Image Processing and Analysis

Lecture 3、Image Enhancement in the Spatial Domain

Fang Wan
School of Computer Science and Technology, UCAS

October 11, 2025

Preview

- The principal objective of enhancement is to process an image so that the result is more suitable than the original image for a specific application.
  - ”Specific” means the techniques are very much problem oriented.
- Image enhancement approaches fall into two broad categories, spatial domain methods and frequency domain method.
  - Spatial domain methods are based on direct manipulation of pixels in an image.
  - Frequency domain techniques are based on modifying the Fourier transform of an image.
- There is no general theory of image enhancement.
  - When an image is processed for visual interpretation, the viewer is the ultimate judge of how well a particular method works, which makes it difficult to compare the performance of different methods.
  - For machine perception, the evaluation task is somewhat easier, e.g., character recognition task.

Outline

Background

- A mathematical representation of spatial domain processing:

$$g(x, y) = T[f(x, y)],$$

where $f(x, y)$: the input image
$g(x, y)$: the processed image
T: an operator on f, defined over some neighborhood of (x, y)

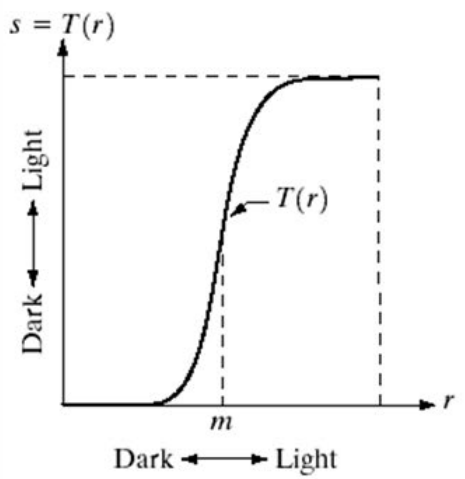
  - T can operate on a set of images, e.g., noise reduction
  - Square and rectangular neighborhood are by far the most popular due to their ease of implementation, although circle is also used.

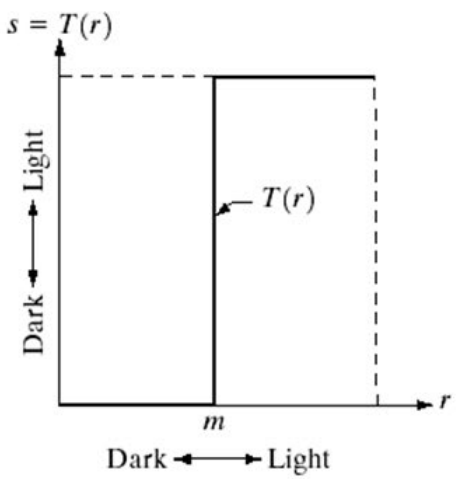
- The simplest form of T is when the neighborhood is of size 1×1. In this case, g depends only on the value of f at (x, y), i.e.,

$$s = T(r),$$

that is called Intensity Transformation

Gray-level Transformation





Power-Law Transformation

- $s = cr^{\gamma}$ where c, γ: positive constants

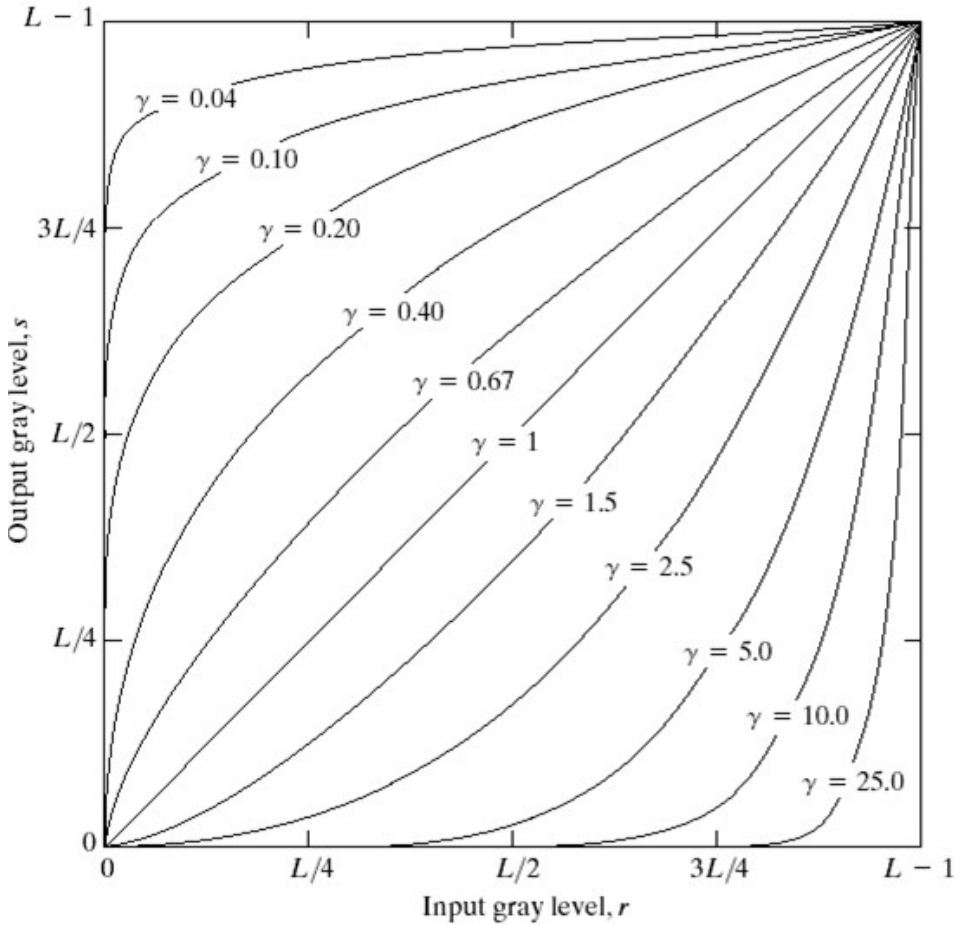
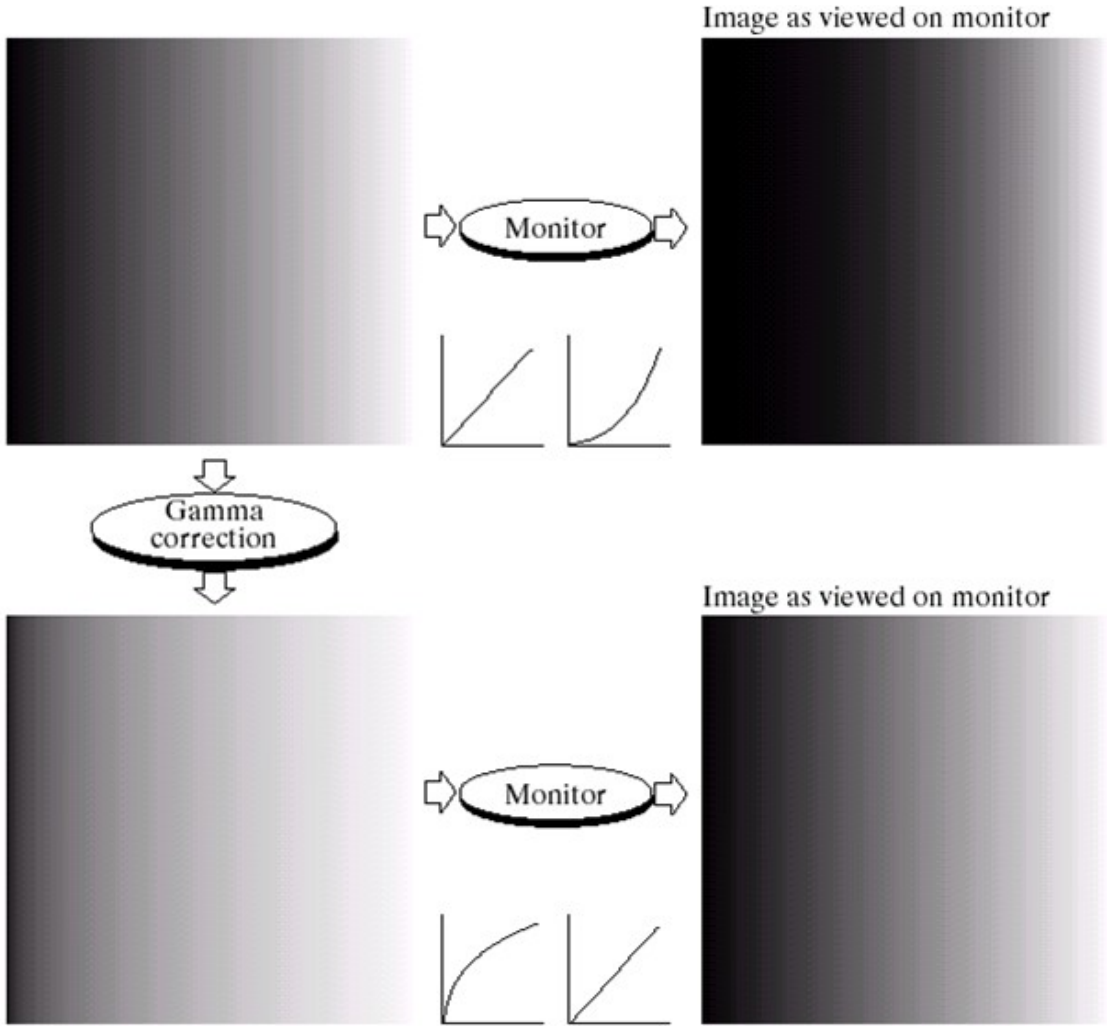


FIGURE 3.6 Plots of the equation $s = cr^{\gamma}$ for various values of γ ($c = 1$ in all cases).

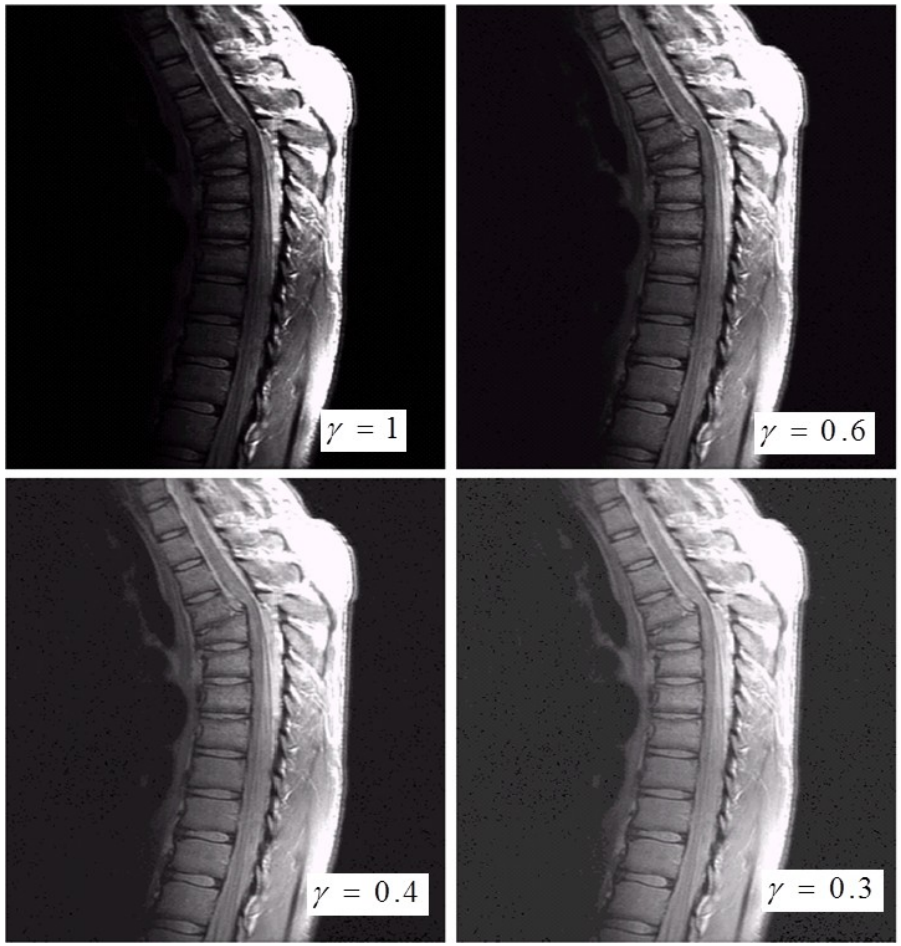
Power-Law Transformation Example 1: Gamma Correction

a b
c d

FIGURE 3.7
(a) Linear-wedge gray-scale image. (b) Response of monitor to linear wedge. (c) Gamma-corrected wedge. (d) Output of monitor.



Power-Law Transform Example 2: Contrast manipulation



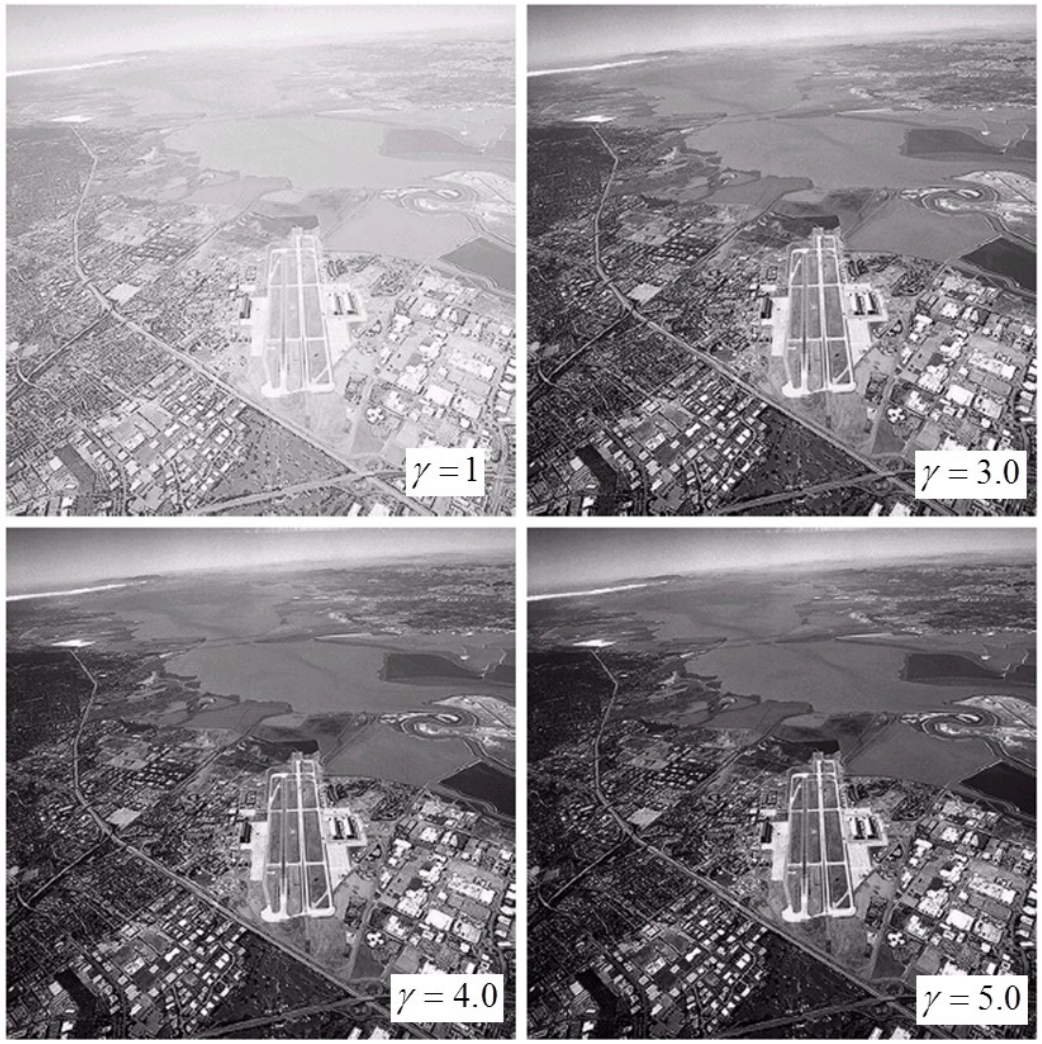
a b
c d

FIGURE 3.8
(a) Magnetic resonance (MR) image of a fractured human spine.
(b)–(d) Results of applying the transformation in Eq. (3.2-3) with $c = 1$ and $\gamma = 0.6, 0.4$, and 0.3, respectively.
(Original image for this example courtesy of Dr. David R. Pickens, Department of Radiology and Radiological Sciences, Vanderbilt University Medical Center.)

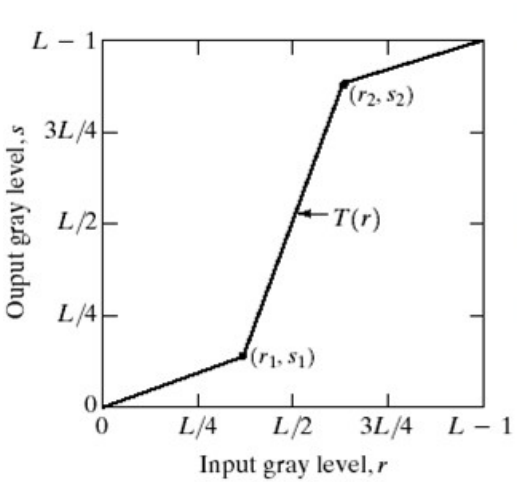
Power-Law Transform Example 3: Contrast manipulation

a b
c d

FIGURE 3.9
(a) Aerial image. (b)–(d) Results of applying the transformation in Eq. (3.2-3) with $c = 1$ and $\gamma = 3.0, 4.0$, and 5.0, respectively. (Original image for this example courtesy of NASA.)



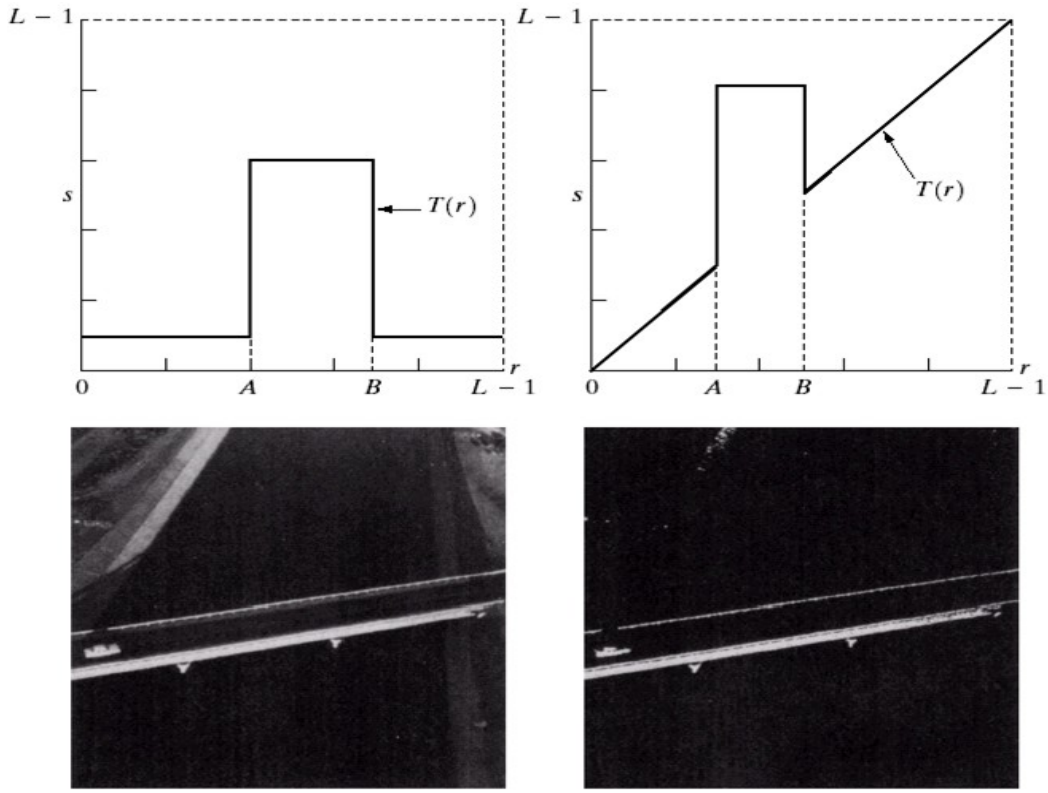
Piecewise-Linear Transformation Functions
Case 1: Contrast Stretching



a b
c d

FIGURE 3.10 Contrast stretching. (a) Form of transformation function. (b) A low-contrast image. (c) Result of contrast stretching. (d) Result of thresholding. (Original image courtesy of Dr. Roger Heady, Research School of Biological Sciences, Australian National University, Canberra, Australia.)

Piecewise-Linear Transformation Functions
Case 2: Gray-level Slicing



a b
c d

FIGURE 3.11
(a) This transformation highlights range $[A, B]$ of gray levels and reduces all others to a constant level. (b) This transformation highlights range $[A, B]$ but preserves all other levels. (c) An image. (d) Result of using the transformation in (a).

Piecewise-Linear Transformation Functions
Case 3: Gray-level Slicing

- Bit-plane slicing:
  - It can highlight the contribution made to total image appearance by specific bits.
  - Each pixel in an image represented by 8 bits.
  - Image is composed of eight 1-bit planes, ranging from bit-plane 0 for the least significant bit to bit plane 7 for the most significant bit.

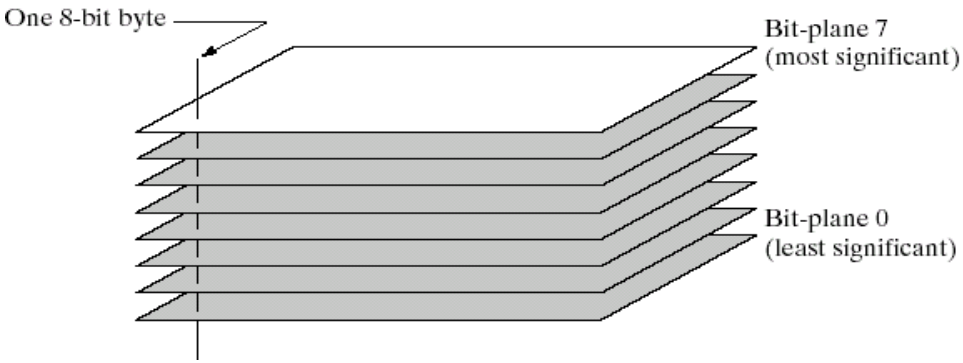


FIGURE 3.12 Bit-plane representation of an 8-bit image.

Piecewise-Linear Transformation Functions
Bit-plane Slicing: A Fractal Image

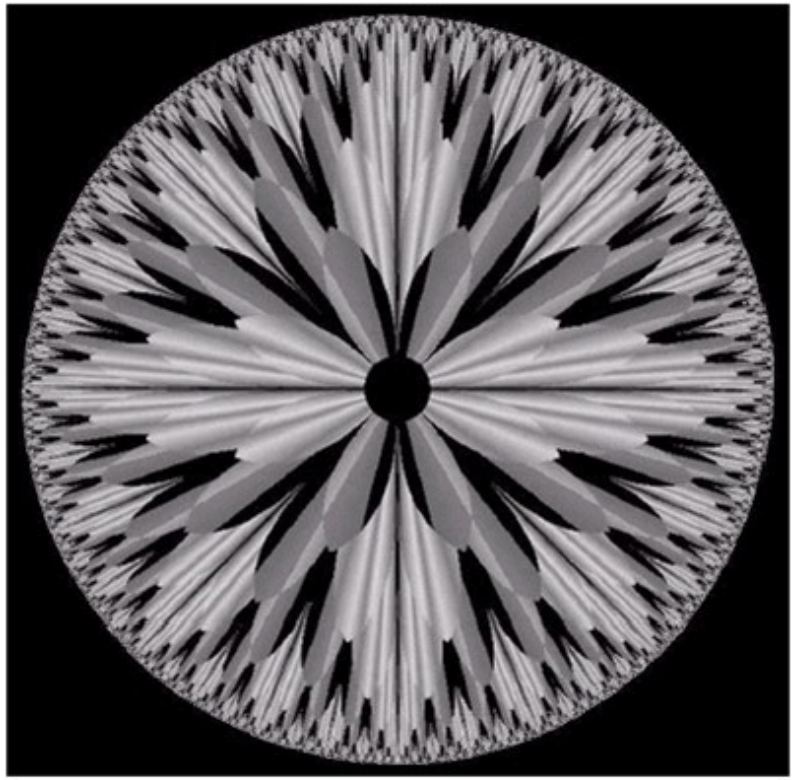


FIGURE 3.13 An 8-bit fractal image. (A fractal is an image generated from mathematical expressions). (Courtesy of Ms. Melissa D. Binde, Swarthmore College, Swarthmore, PA.)

Piecewise-Linear Transformation Functions
Bit-plane Slicing: A Fractal Image

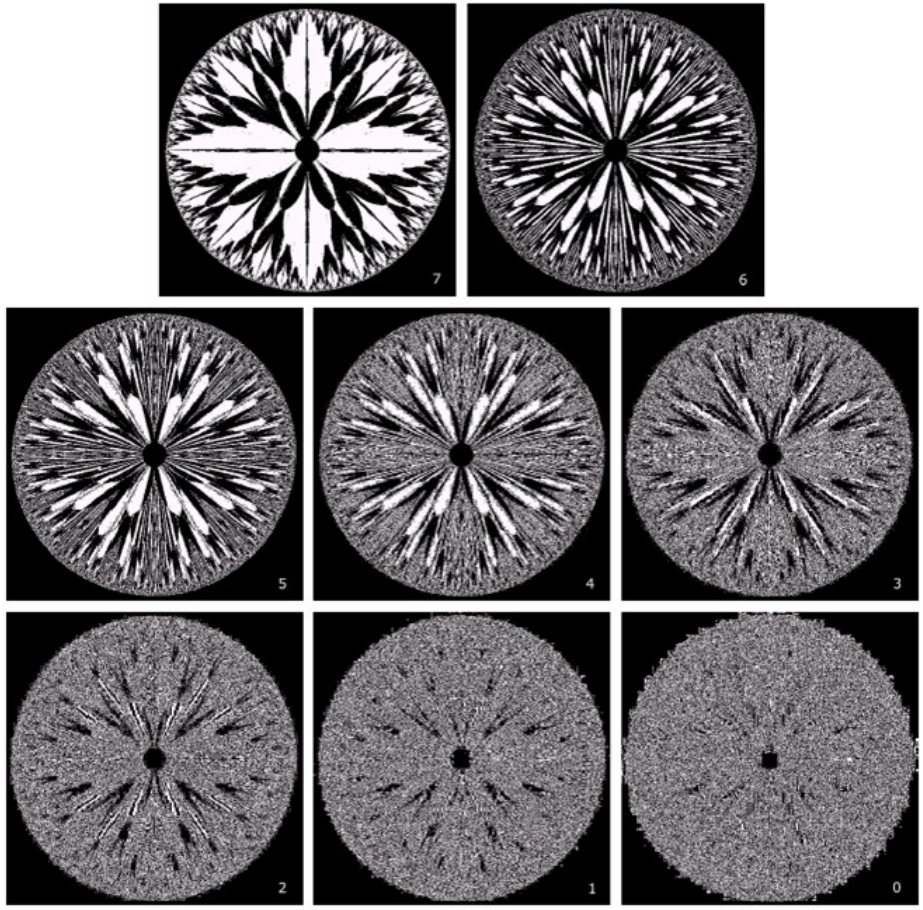
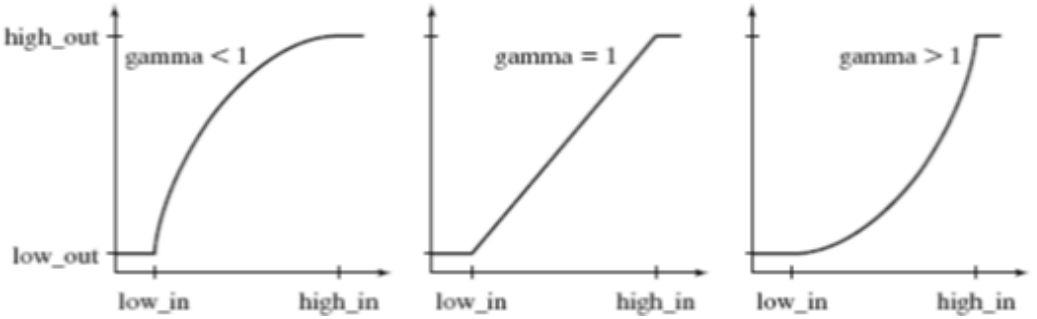


FIGURE 3.14 The eight bit planes of the image in Fig. 3.13. The number at the bottom, right of each image identifies the bit plane.

Function imadjust in IPT

- $g = imadjust(f, [low\_in, high\_in], [low\_out, high\_out], gamma)$
- This function maps the intensity values in image f to new values in g.
- Such that values between low_in and $high\_in$ map to values between low_out and $high\_out$.
- Values below low_in and above $high\_in$ are clipped; that is, values below low_in map to low_out, and those above $high\_in$ map to $high\_out$.

Function imadjust in IPT



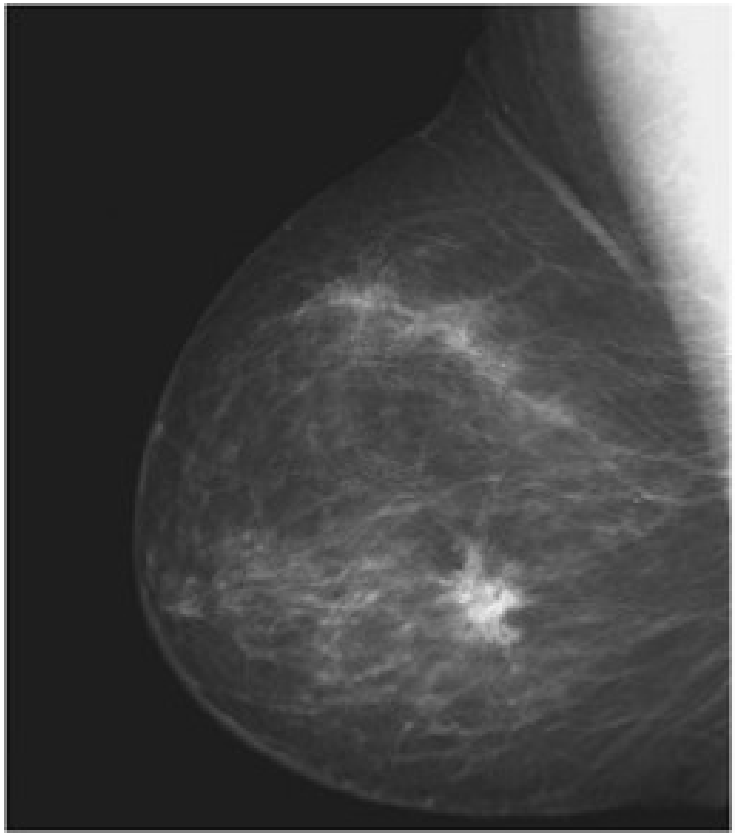
a b c

FIGURE 3.2 The various mappings available in function `imadjust`.

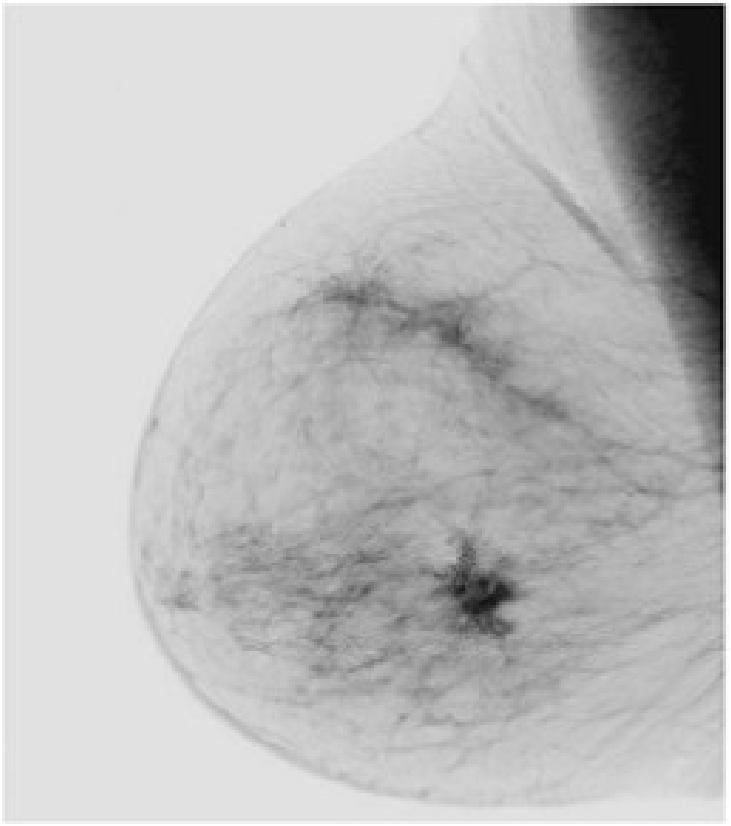
- Parameter *gamma* specifies the shape of the curve that maps the intensity values in f to create g.
- If *gamma* is less than 1, the mapping is weighted toward higher *brighter* output values, as Fig. 3.2(a) shows.
- If *gamma* is greater than 1, the mapping is weighted toward lower *darker* output values.
- If it is omitted from the function argument, *gamma* defaults to 1 *linearmapping*.

An Example

- I=imread('Fig0303(a)(breast).tif');
- imshow(I)

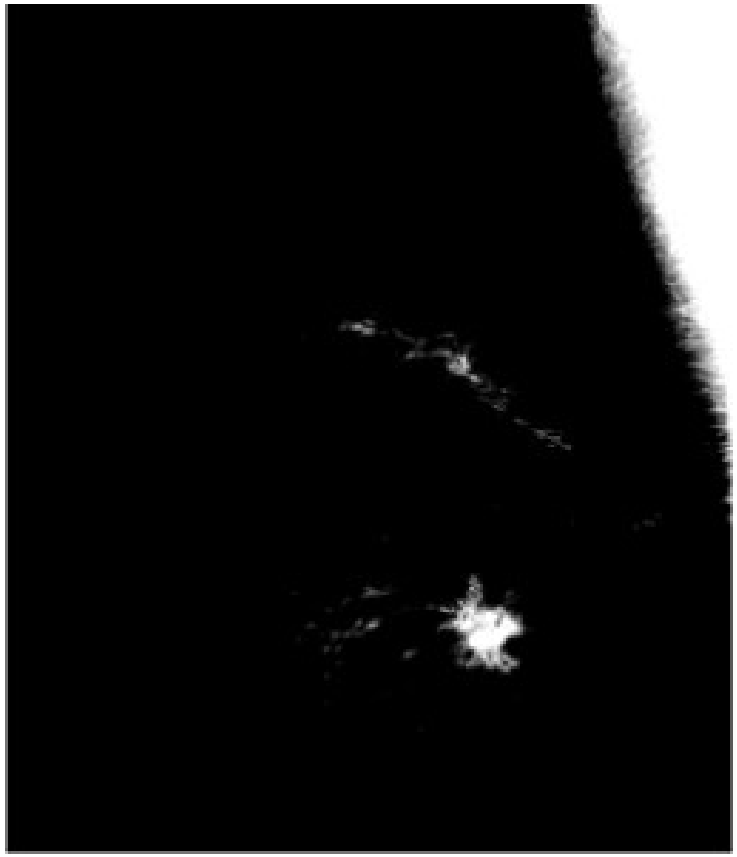


- G=imadjust(I,[0 1],[1 0]);
- imshow(G)

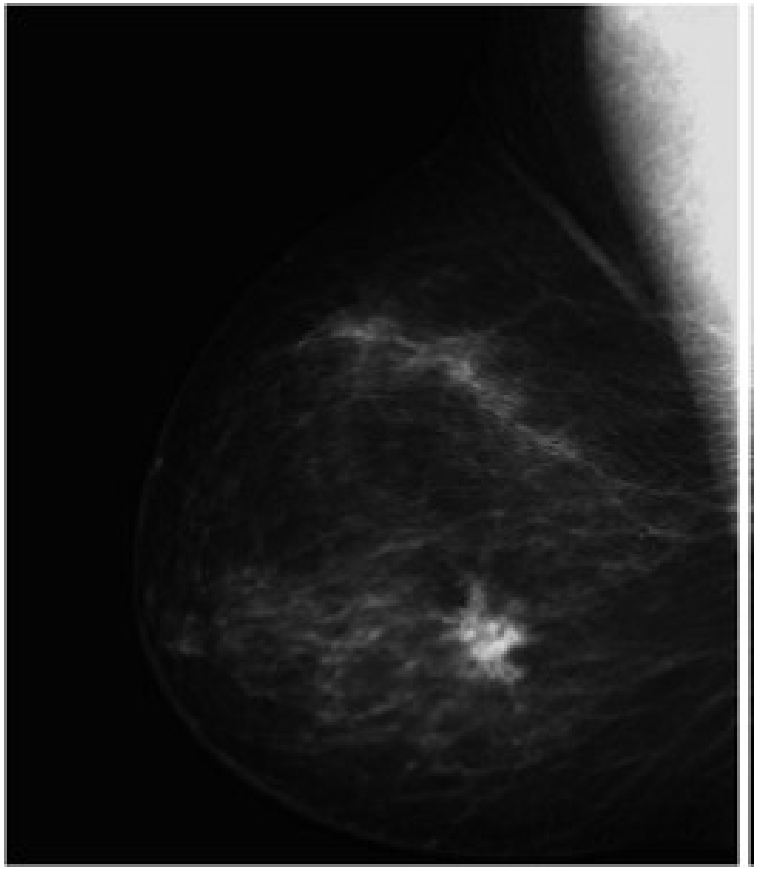


An Example

- G=imadjust(I,[0.5 0.75],[0 1]);
- imshow(G)



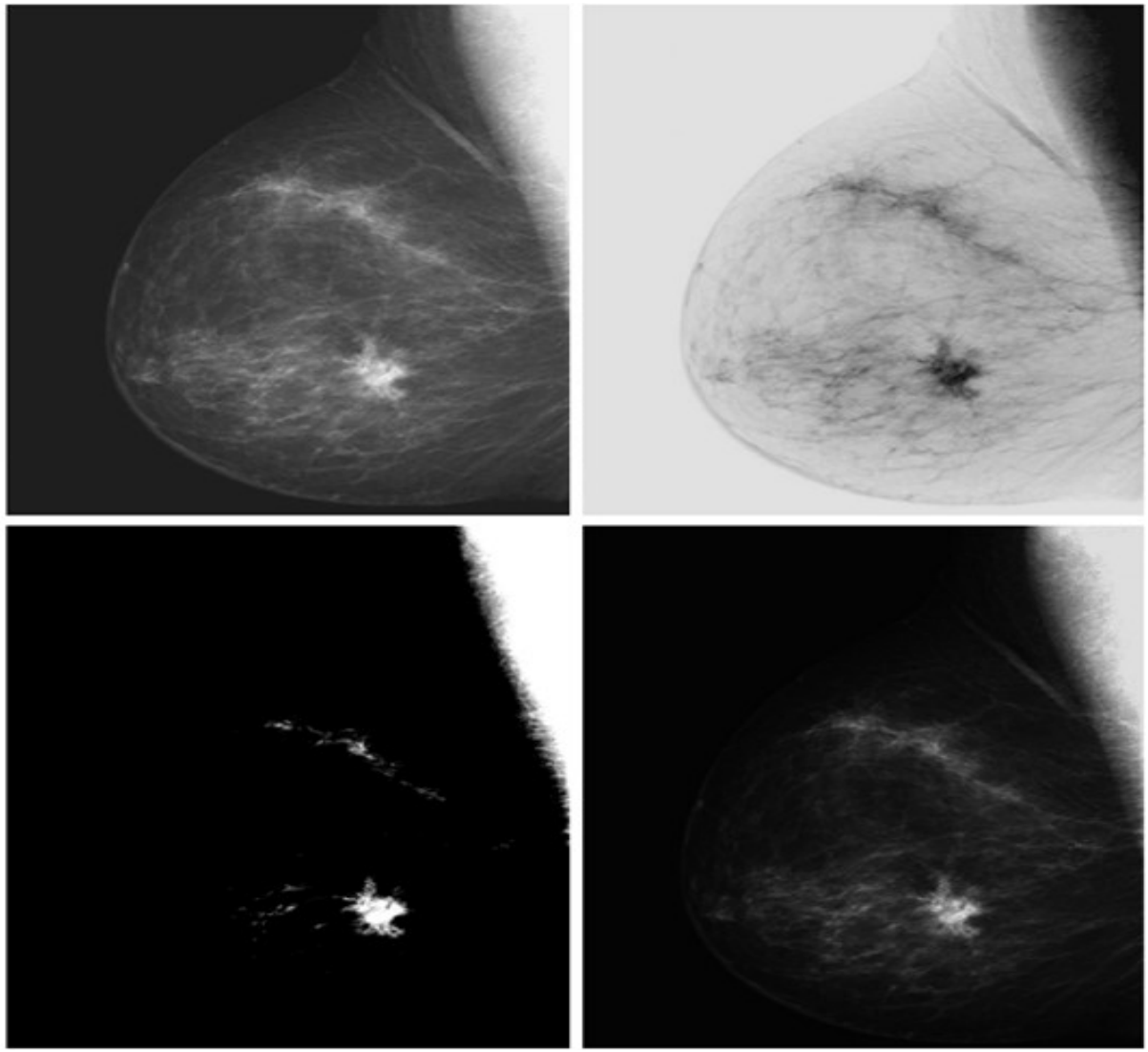
- G=imadjust(I,[],[],2);
- imshow(G)



Function imadjust

a b
c d

FIGURE 3.3 (a) Original digital mammogram. (b) Negative image. (c) Result of expanding the intensity range [0.5, 0.75]. (d) Result of enhancing the image with `gamma = 2`. (Original image courtesy of G. E. Medical Systems.)



Logarithmic Transformation

- Logarithmic and contrast-stretching transformations are basic tools for dynamic range manipulation.
- Logarithm transformations are implemented using the expression

$$s = c\log(1 + r)$$

  - where c is a constant.
  - The shape of this transformation is similar to the $gamma$ curve with the low values set at 0 and the high values set to 1 on both scales.
  - Note, however, that the shape of the $gamma$ curve is variable,whereas the shape of the log function is fixed.

Logarithmic Transformation

- One of the principal uses of the log transformation is to compress dynamic range.
  - For example, it is not unusual to have a Fourier spectrum (Chapter 4) with values in the range $[0, 10^6]$ or higher. When displayed on a monitor that is scaled linearly to 8 bits, the high values dominate the display, resulting in lost visual detail for the lower intensity values in the spectrum. By computing the log, a dynamic range on the order of, for example, 10^6 is reduced to approximately 14, which is much more manageable.
- When performing a logarithmic transformation, it is often desirable to bring the resulting compressed values back to the full range of the display. For 8 bits, the easiest way to do this in MATLAB is with the statement

$$gs = im2uint8(mat2gray(g));$$

  - Use of $mat2gray$ brings the values to the range [0, 1] and $im2uint8$ brings them to the range [0, 255].

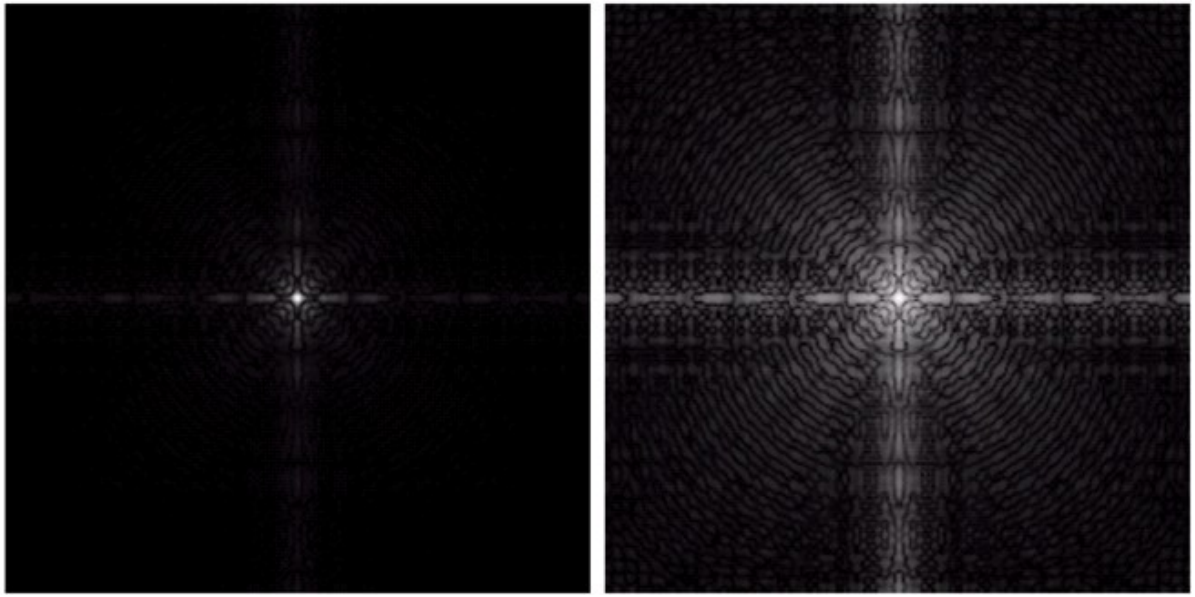
Logarithmic Transformation

- Figure 3.5(a) is a Fourier spectrum with values in the range 0 to displayed on a linearly scaled, 8-bit system. Figure 3.5(b) shows the result obtained using the commands

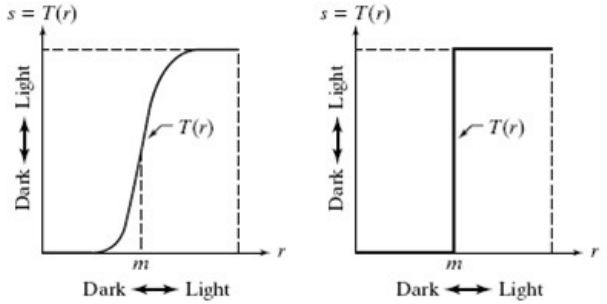
```
>> g = im2uint8(mat2gray(log(1 + double(f))));
>> imshow(g)
```

a b

FIGURE 3.5
(a) Fourier spectrum.
(b) Result of applying the log transformation given in Eq. (3.2-2) with $c = 1$.



Contrast-Stretching Transformation



a b

FIGURE 3.4
(a) Contrast-stretching transformation. (b) Thresholding transformation.

- A contrast-stretching transformation function can be defined as

$$s = T(r) = \frac{1}{1 + (m/r)^E}$$

  - it compresses the input levels lower than m into a narrow range of dark levels in the output image;
  - similarly, it compresses the values above m into a narrow band of light levels in the output;
  - where r represents the intensities of the input image, s the corresponding intensity values in the output image, and E controls the slope of the function.
- This equation is implemented in MATLAB for an entire image as

```
g = 1./(1 + (m./(double(f) + eps)).^E)
```

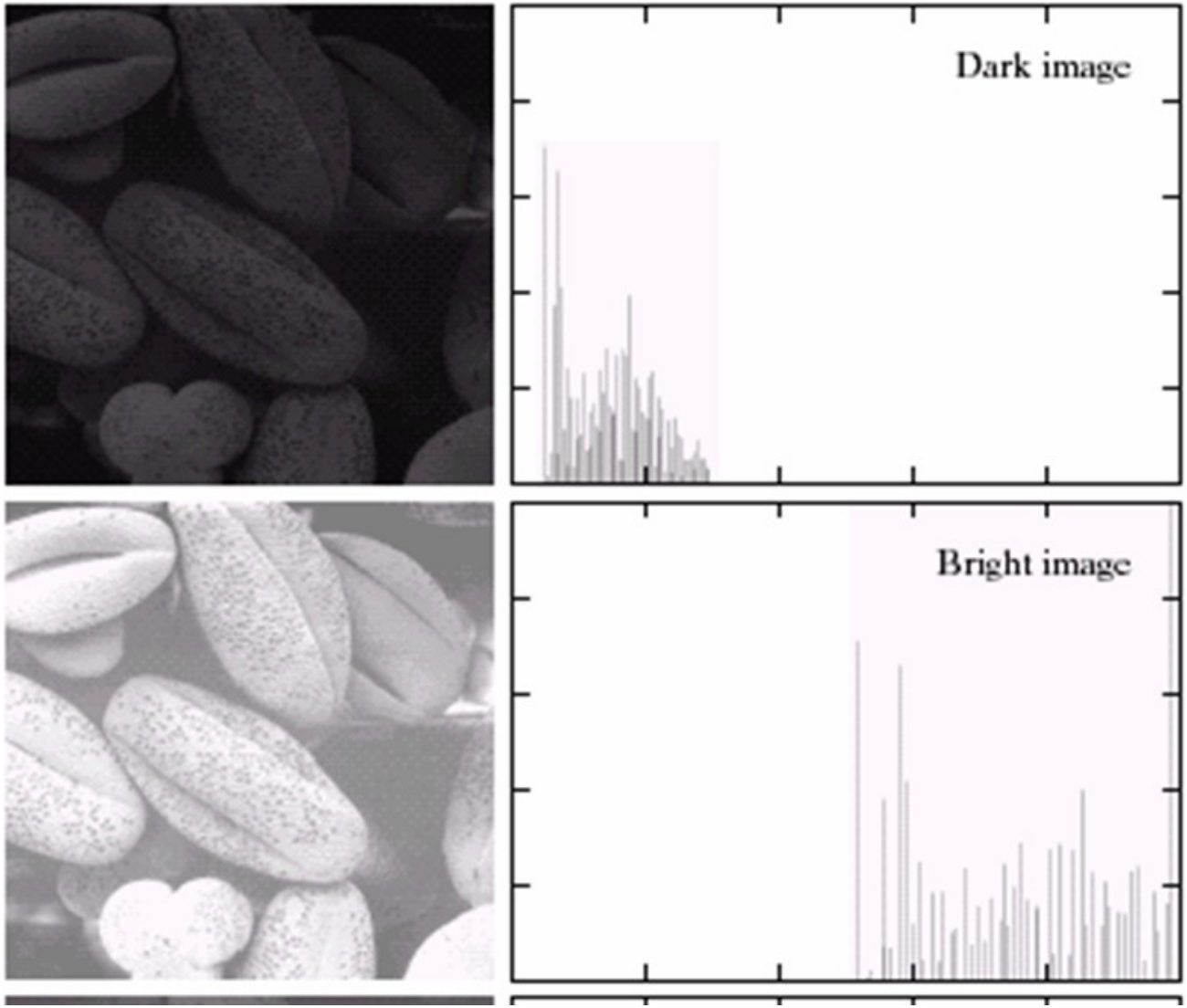
Histogram Processing

- Intensity transformation are based on information extracted from image intensity histograms, and histogram plays a basic role in image processing, in areas such as enhancement, compression, segmentation, and description.
- The histogram of a digital image with L total possible intensity levels in the range [0, G] is defined as the discrete function

$$h(r_k) = n_k$$

  - where r_k is the kth intensity level in the interval [0, G] and n_k is the number of pixels in the image whose intensity level is r_k.The value of G is 255 for images of class uint8, 65535 for images of class uint16, and 1.0 for images of class double.

Generating and Plotting Image Histograms



Normalized Histogram

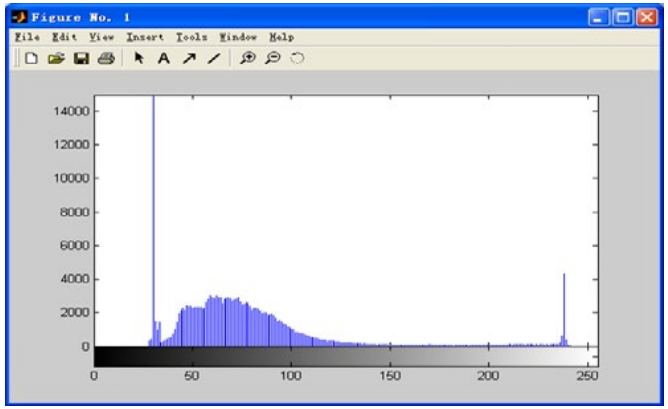
- Through dividing all elements of $h(r_k)$ by the total number of pixels in the image, a normalized histogram can be obtained, i.e.,

$$p(r_k) = \frac{h(r_k)}{N} = \frac{n_k}{N}, N = \sum_{k=0}^{L-1} n_k, k = 0,1, \dots, L-1$$

- From basic probability, we know $p(r_k)$ can be considered as an estimate of the probability of occurrence of intensity level r_k.

Generating Image Histograms

- h=imhist(f,b)
  - Where f is the input image, h is its histogram, and b is the number of bins used in forming the histogram.
  - If b is not included in the argument, b = 256 is used by default.
  - A bin is simply a subdivision of the intensity scale.
- We obtain the normalized histogram simply by using the expression:
  p=imhist(f,b)/numel(f)
- An example
  - f=imread('Fig3_8_a.tif');
  - imhist(f)

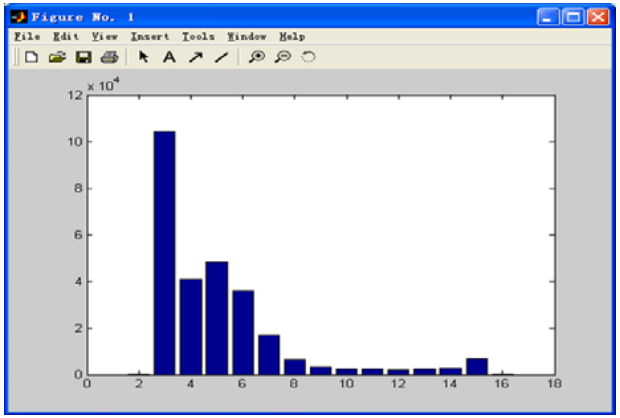


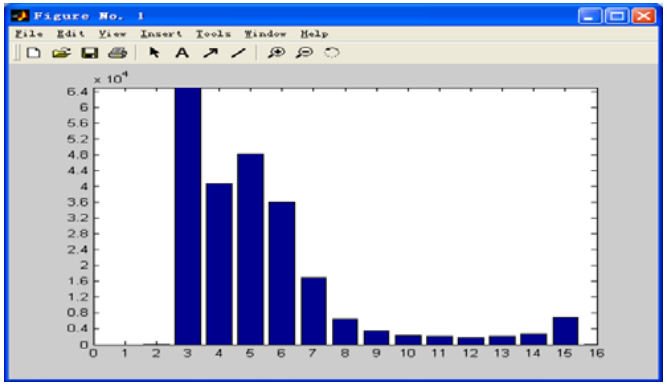
Plotting Image Histograms

- Histograms often are plotted using bar graphs. For this purpose we can use the function bar(horz, v, width)
  - Where v is a row vector containing the points to be plotted.
  - horz is a vector of the same dimension as v that contains the increments of the horizontal scale. If horz is omitted, the horizontal axis is divided in units from 0 to length(v).
  - width is a number between 0 and 1.

- s=imread('Fig0303(a)(breast).tif');
- h1=imhist(s,16);
- horz=1:16;
- bar(horz,h1,0.8)

- axis([0 16 0 65000]);
- set(gca,'xtick',0:1:16);
- set(gca,'ytick',0:4000:65000);





Plotting Image Histograms using stem

- A stem graph is similar to a bar graph. The syntax is

  stem(horz, v, 'color_linestyle_marker', 'fill')

  - where v is row vector containing the points to be plotted, and horz is as described for bar.
  - *color_linestyle_marker* is a triplet of values from Table below.

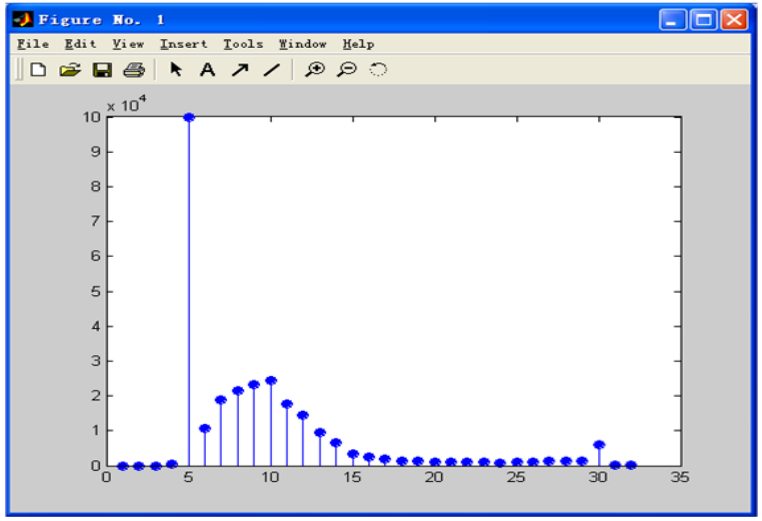
Symbol	Color	Symbol	Line Style	Symbol	Marker
k	Black	–	Solid	+	Plus sign
w	White	--	Dashed	o	Circle
r	Red	:	Dotted	*	Asterisk
g	Green	-.	Dash-dot	.	Point
b	Blue	none	No line	x	Cross
c	Cyan			s	Square
y	Yellow			d	Diamond
m	Magenta			none	No marker

TABLE 3.1 Attributes for functions `stem` and `plot`. The `none` attribute is applicable only to function `plot`, and must be specified individually. See the syntax for function `plot` below.

Some examples using stem

- stem(v, 'rs')
  produces a stem plot where the lines and markers are red, the lines are dashed, and the markers are squares.
- If *fill* is used, and the *marker* is a circle, square, or diamond, the *marker* is filled with the color specified in color.
- The default *color* is black, the *linestyle* default is solid, and the default *marker* is a circle.

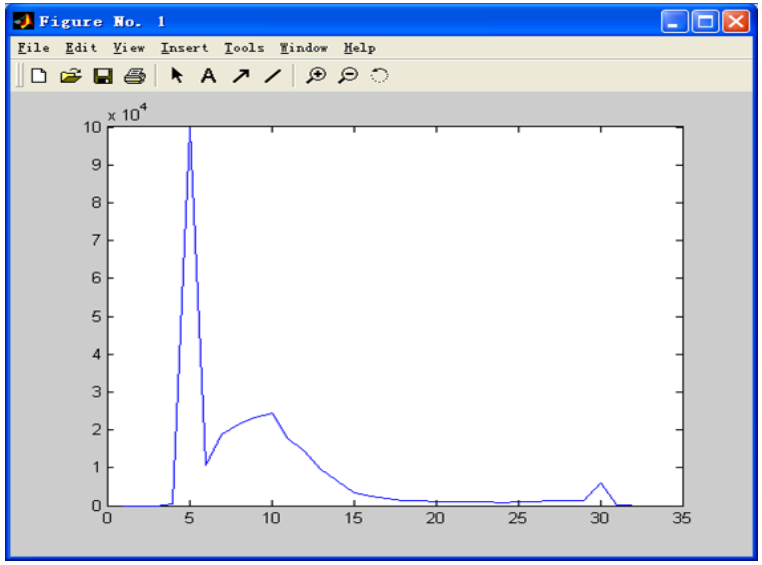
- s=imread('Fig0303(a)(breast).tif');
- hi=imhist(s,32);
- stem(hi,'b-o','fill');



Plotting Image Histograms using plot

- Function *plot* plots a set of points by linking them with straight lines. The syntax is plot(horz, v, 'color_linestyle_marker')
- where the arguments are as defined previously for stem plots.
- Function *plot* is used frequently to display transformation functions.

- s=imread('Fig0303(a)(breast).tif');
- hi=imhist(s,32);
- plot(hi);



Histogram Equalization

- Histogram equalization:
  - To improve the contrast of an image.
  - To transform an image in such a way that the transformed image has a nearly uniform distribution of pixel values.
- Transformation:
  - Assume r has been normalized to the interval [0,1], with $r = 0$ representing black and $r = 1$ representing white

$$s = T(r), 0 \leq r \leq 1$$

  - The transformation function satisfies the following conditions:
    - $T(r)$is single-valued and monotonically increasing in the interval $0 \leq r \leq 1$
    - $0 \leq T(r) \leq 1$ for $0 \leq r \leq 1$

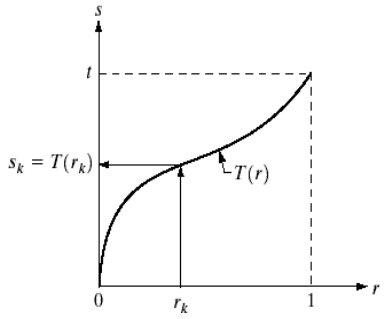


FIGURE 3.16 A gray-level transformation function that is both single valued and monotonically increasing.

Histogram Equalization

- Histogram equalization is based on a transformation of the probability density function of a random variable.
- Let $p_r(r)$ and $p_s(s)$ denote the probability density function of random variable r and s, respectively.
- If $p_r(r)$ and $T(r)$ are known, then the probability density function $p_s(s)$ of the transformed variable s can be obtained

$$p_s(s) = p_r(r)\left|\frac{dr}{ds}\right|$$

- Define a transformation function

$$s = T(r) = \int_0^r p_r(w)dw$$

  - where w is a dummy variable of integration and the right side of this equation is the cumulative distribution function of random variable r.

Histogram Equalization

- Given transformation function $T(r)$

$$s = T(r) = \int_0^r p_r(w)dw$$

$$\frac{ds}{dr} = \frac{dT(r)}{dr} = \frac{d\left[\int_0^r p_r(w)dw\right]}{dr} = p_r(r)$$

$$p_s(s) = p_r(r)\left|\frac{dr}{ds}\right| = p_r(r)\frac{1}{p_r(r)} = 1, 0 \le s \le 1$$

- $p_s(s)$ now is a uniform probability density function.
- $T(r)$ depends on $p_r(r)$, but the resulting $p_s(s)$ always is uniform.

Histogram Equalization

- In discrete version:
  - The probability of occurrence of gray level r_k in an image is

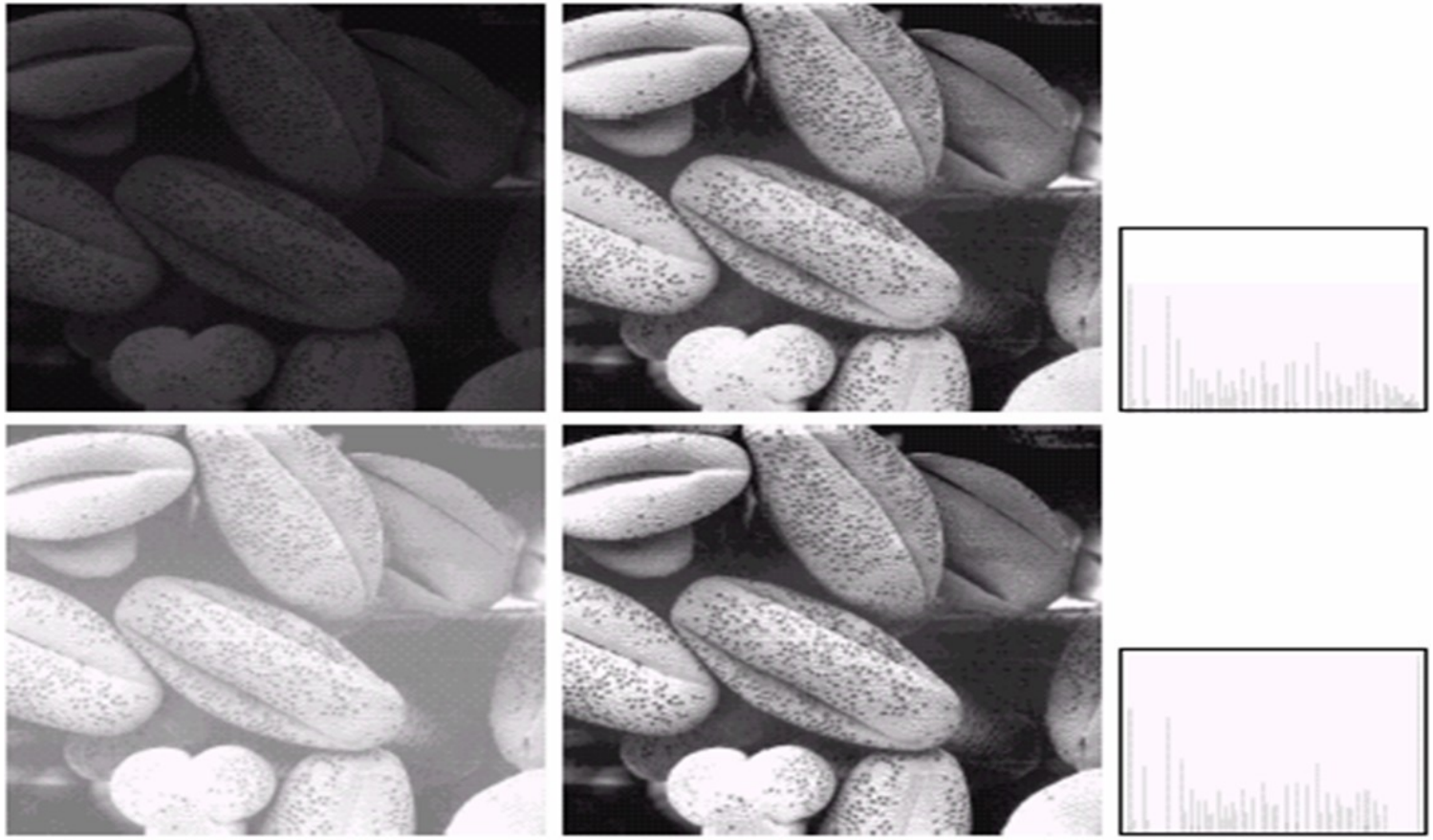
$$p_r(r_k) = \frac{n_k}{N}, k = 0,1, \ldots, L-1$$

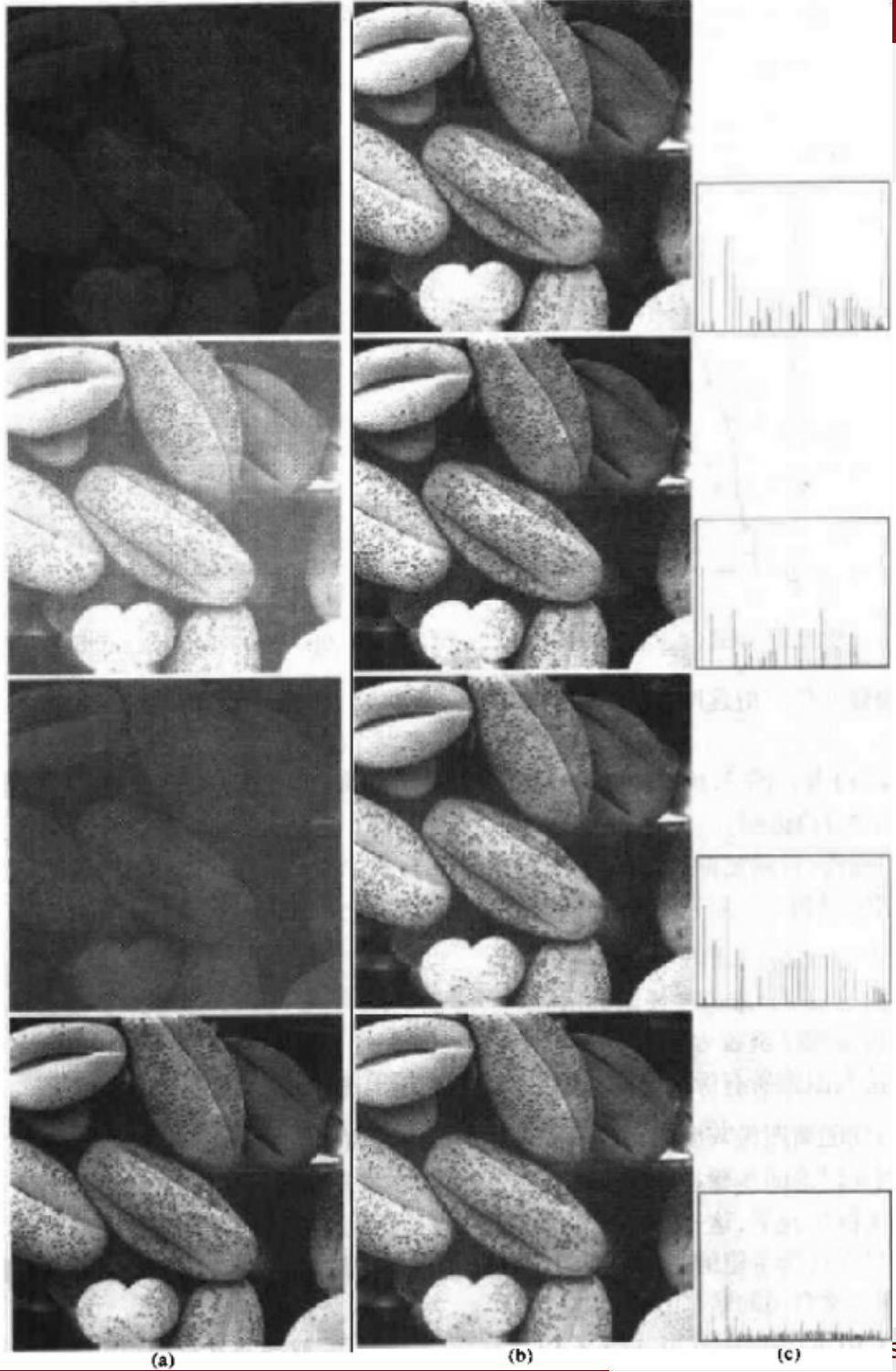
  - The transformation function is

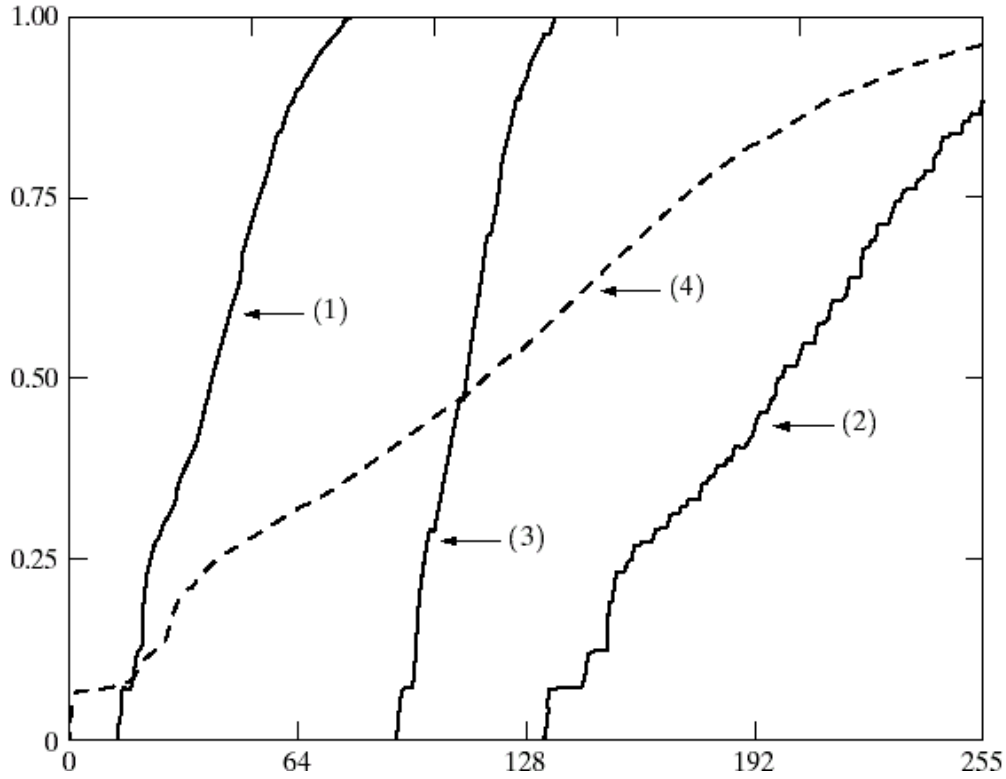
$$s = T(r_k) = \sum_{j=0}^{k} p_r(r_j) = \sum_{j=0}^{k} \frac{n_j}{N}$$

  - Thus, an output image is obtained by mapping each pixel with level r_k in the input image into a corresponding pixel with level s_k.

Histogram Equalization




(a)
(b)
(c)


1.00
0.75
0.50
0.25
0
0
64
128
192
255
(1)
(2)
(3)
(4)

Histogram Equalization

- Histogram equalization is implemented in the toolbox by function histeq, which has the syntax

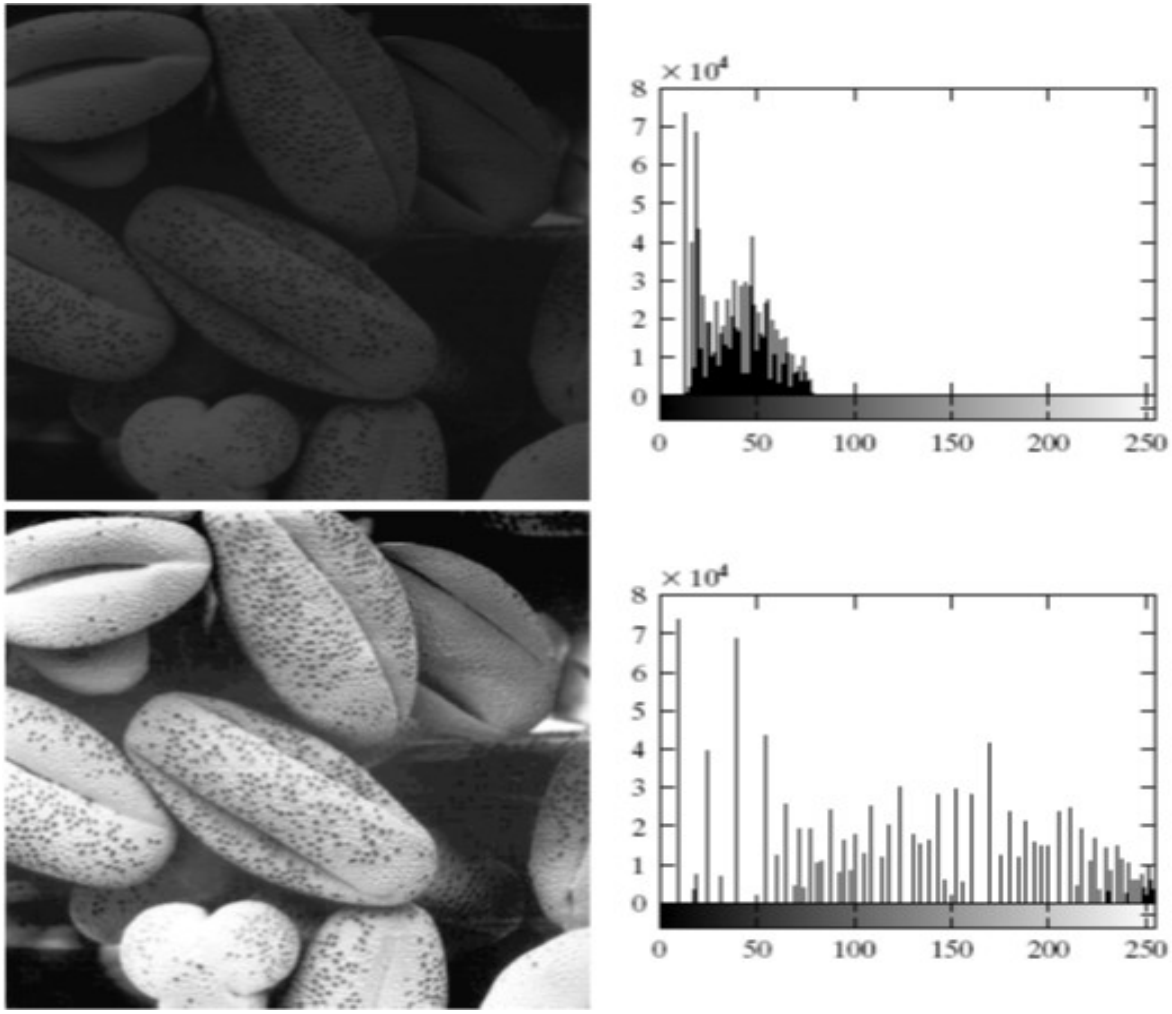
$$g = \text{histeq}(f, \text{nlev})$$

  - where f is the input image and nlev is the number of intensity levels specified for the output image.
  - If nlev is equal to L (the total number of possible levels in the input image), then histeq implements the transformation function directly. If nlev is less than L, then histeq attempts to distribute the levels so that they will approximate a flat histogram.

Histogram Equalization in Matlab

- >> f = imread('Fig0308(a)(pollen).tif');
- >> imshow(f);
- >> figure,imhist(f);
- >> ylim('auto');
- >> g = histeq(f,256);
- >> figure, imshow(g);
- >> figure, imhist(g);
- >> ylim('auto');

Histogram Equalization in Matlab

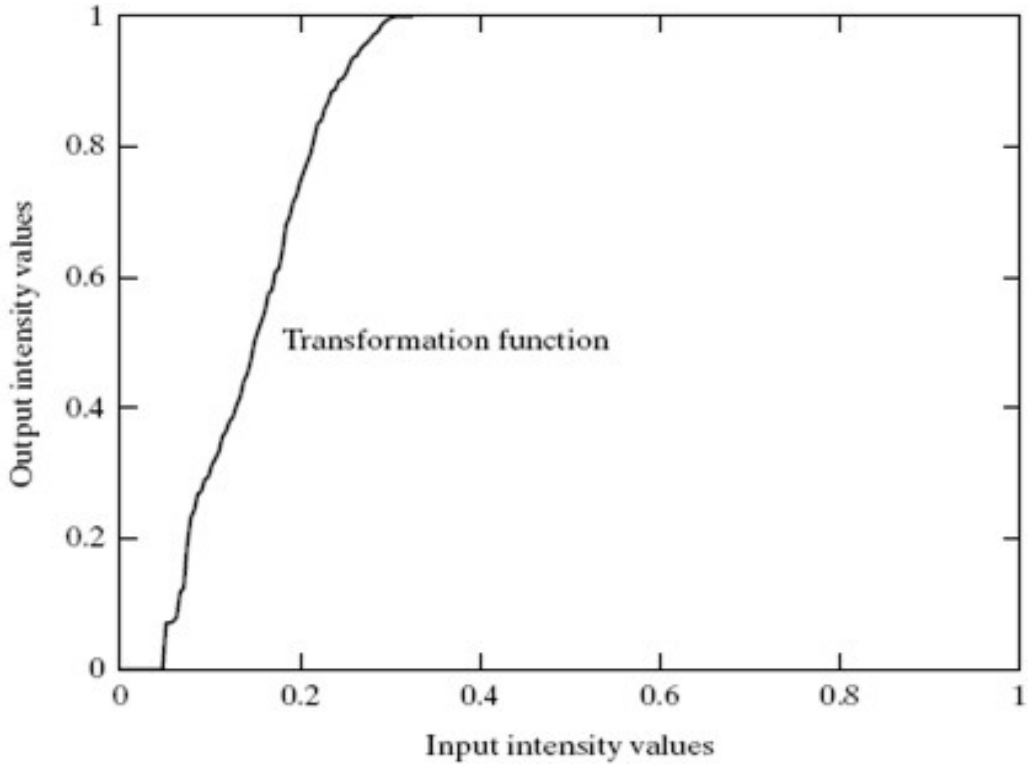


a b
c d

FIGURE 3.8 Illustration of histogram equalization. (a) Input image, and (b) its histogram. (c) Histogram-equalized image, and (d) its histogram. The improvement between (a) and (c) is quite visible. (Original image courtesy of Dr. Roger Heady, Research School of Biological Sciences, Australian National University, Canberra.)

Histogram Equalization

FIGURE 3.9 Transformation function used to map the intensity values from the input image in Fig. 3.8(a) to the values of the output image in Fig. 3.8(c).



Histogram Matching

- Histogram matching is similar to histogram equalization, except that instead of trying to make the output image have a flat histogram, we would like it to have a histogram of a specified shape.
- Consider for a moment continuous levels that are normalized to the interval [0, 1], and let r and z denote the intensity levels of the input and output images. The input levels have probability density function $p_r(r)$ and the output levels have the specified probability density function $p_z(z)$.

Histogram Matching

- We know from the discussion in the previous section that the transformation:

$$s = T(r) = \int_0^r p_r(w)dw$$

result in a ideal equalized histogram $p_s(s)$.

- Suppose now we define a variable z with the property

$$H(z) = \int_0^z p_z(w)dw = s$$

- From the preceding two equations, it follows that

$$z = H^{-1}(s) = H^{-1}(T(r));$$

- We can find $T(r)$ from the input image (this is the histogram equalization transformation discussed in the previous section), so it follows that we can use the preceding equation to find the transformed levels z whose PDF is the specified $p_z(z)$ as long as we can find H^{-1}.

Histogram Matching

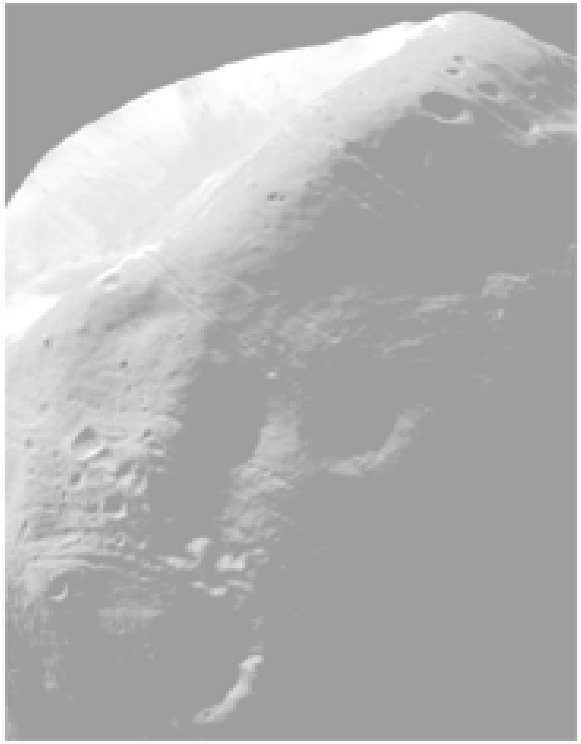
- The toolbox implements histogram matching using the following syntax in histeq:

$$g = \text{histeq}(f, \text{hspec})$$

  - where f is the input image, hspec is the specified histogram (a row vector of specified values), and g is the output image, whose histogram approximates the specified histogram, hspec.
- This vector should contain integer counts corresponding to equally spaced bins. A property of histeq is that the histogram of g generally better matches hspec when length (hspec) is much smaller than the number of intensity levels in f.

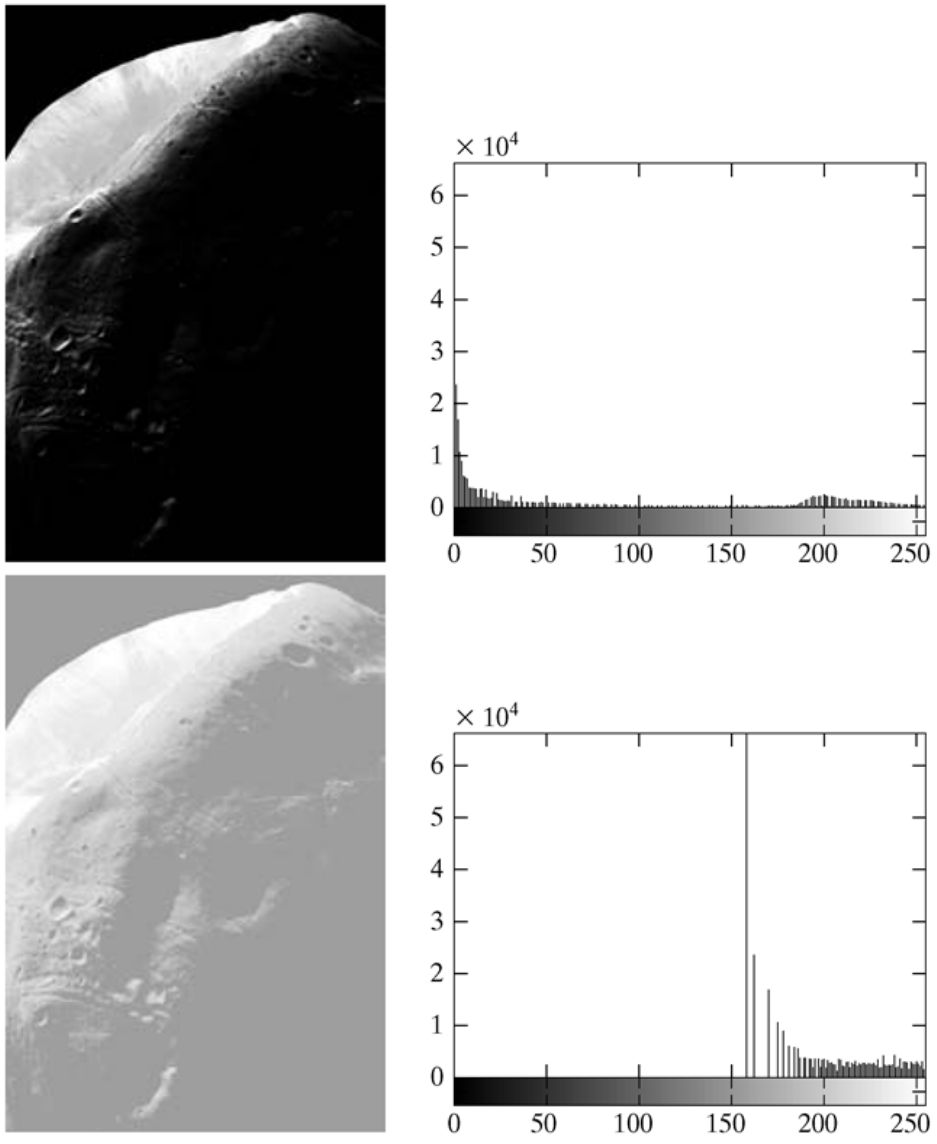
Histogram Matching

- f=imread('Fig0310(a)(Moon Phobos).tif');
- f1=histeq(f,256);
- imshow(f1);



- It shows that histogram equalization in fact did not produce a particularly good result in this case.
- The reason for this can be seen by studying the histogram of the equalized image.

Histogram Matching



a b
c d

FIGURE 3.10
(a) Image of the Mars moon Phobos.
(b) Histogram.
(c) Histogram-equalized image.
(d) Histogram of (c).
(Original image courtesy of NASA).

Histogram Matching

- One possibility for remedying this situation is to use histogram matching, with the desired histogram having a lesser concentration of components in the low end of the gray scale, and maintaining the general shape of the histogram of the original image.
- We note that the histogram of original image is basically bimodal, with one large mode at the origin, and another, smaller, mode at the high end of the gray scale. These types of histograms can be modeled, for example, by using multimodal Gaussian functions.

Histogram Matching

- The following M-function computes a bimodal Gaussian function normalized to unit area, so it can be used as a specified histogram.
  - Function twomodegauss:

$$p(x) = k + \frac{A_1}{\sqrt{2\pi}\sigma_1}\exp\left(-\frac{(x-m_1)^2}{2\sigma_1^2}\right) + \frac{A_2}{\sqrt{2\pi}\sigma_2}\exp\left(-\frac{(x-m_2)^2}{2\sigma_2^2}\right)$$

- The following interactive function accepts inputs from a keyboard and plots the resulting Gaussian function. Refer to Section 2.10.5 for an explanation of the functions input and str2num. Note how the limits of the plots are set.
  - Function manualhist

Histogram Matching

- Since the problem with histogram equalization in this example is due primarily to a large concentration of pixels in the original image with levels near 0,a reasonable approach is to modify the histogram of that image so that it does not have this property.
- Figure 3.11(a) shows a plot of a function that preserves the general shape of the original histogram, but has a smoother transition of levels in the dark region of the intensity scale. The output of the program, p, consists of 256 equally spaced points from this function and is the desired specified histogram.

Histogram Matching

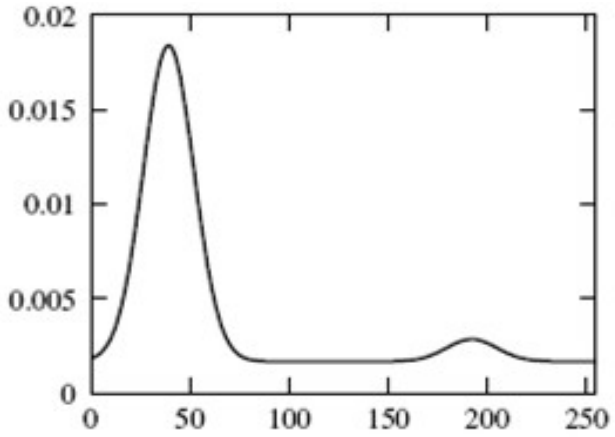
- An image with the specified histogram was generated using the command

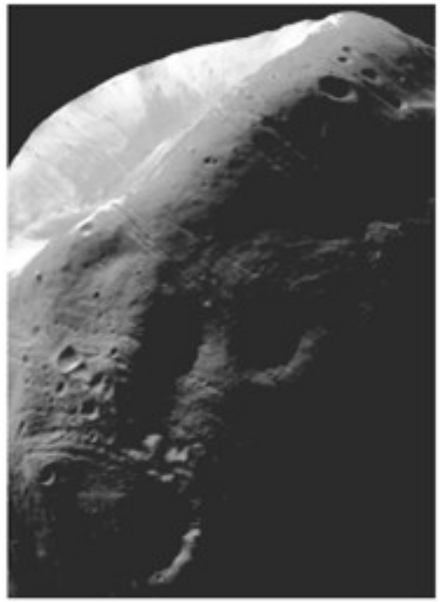
```
>> g = histeq(f, p);
 all commands are:
      >> f=imread('Fig0310(a)(Moon Phobos).tif');
      >> g=histeq(f,manualhist);
      Enter m1, sig1, m2, sig2, A1, A2, k OR x to quit:x;
      >> imshow(g);
```

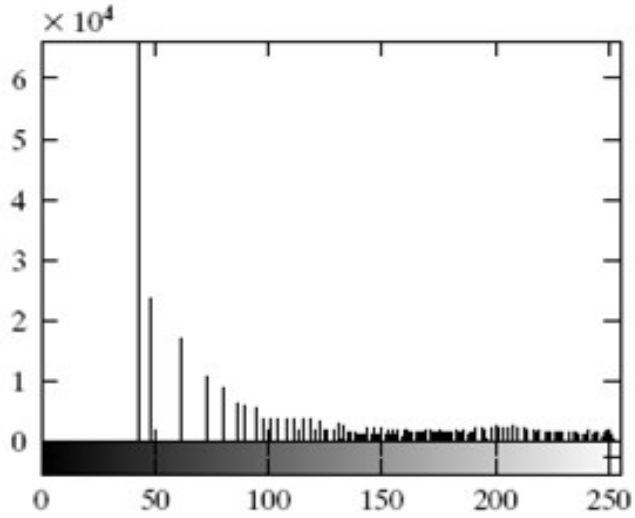
Histogram Matching

a b
c

FIGURE 3.11
(a) Specified histogram.
(b) Result of enhancement by histogram matching.
(c) Histogram of (b).







Spatial Filtering

- In spatial filtering (vs. frequency domain filtering), the output image is computed directly by simple calculations on the pixels of the input image.
- Spatial filtering can be either linear or non-linear.
- For each output pixel, some neighborhood of input pixels is used in the computation.

Linear Spatial Filtering

- In general, linear filtering of an image f of size $M \times N$ with a filter mask of size $m \times n$ is given by

$$g(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) f(x+s, y+t)$$

where $a = (m-1)/2$ and $b = (n-1)/2$

- This concept is called convolution. Filter masks are sometimes called convolution masks or convolution kernels.

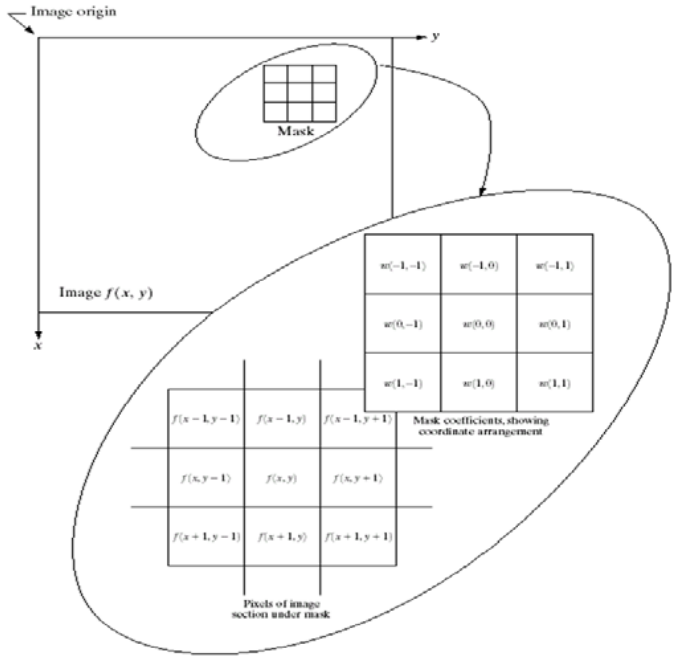


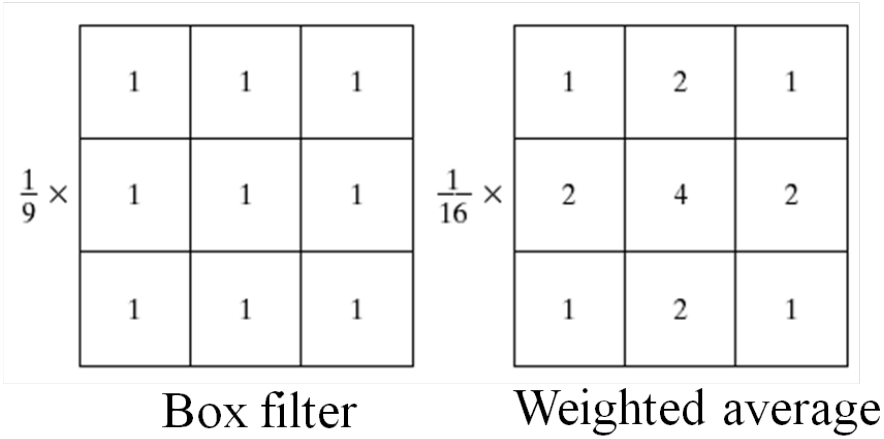
FIGURE 3.32 The mechanics of spatial filtering. The magnified drawing shows a 3×3 mask and the image section directly under it; the image section is shown displaced out from under the mask for ease of readability.

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

FIGURE 3.33 Another representation of a general 3×3 spatial filter mask.

Linear Spatial Filtering(cont.)

- Smoothing linear filters
  - Averaging filters (Lowpass filters in Chapter 4))
    - Box filter
    - Weighted average filter



a b

FIGURE 3.34 Two 3 × 3 smoothing (averaging) filter masks. The constant multiplier in front of each mask is equal to the sum of the values of its coefficients, as is required to compute an average.

Linear Spatial Filtering(cont.)

- There are two closely related concepts that must be understood clearly when performing linear spatial filtering. One is correlation; the other is convolution.
- Correlation is the process of passing the mask ω by the image array f in the manner described in Fig. 3.32.
- Mechanically, convolution is the same process, except that ω is rotated by 180 degree prior to passing it by f.
- These two concepts are best explained by some simple examples.

Linear Spatial Filtering(cont.)

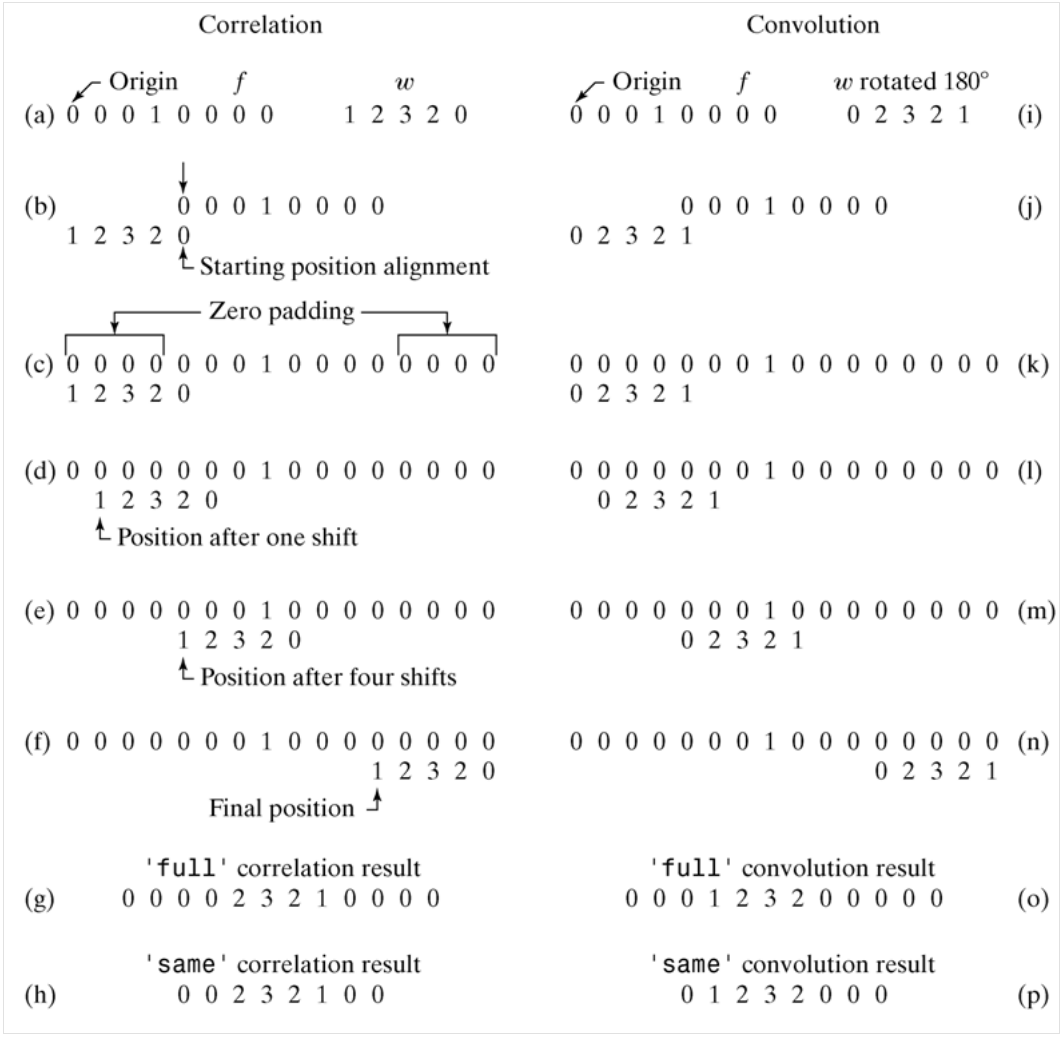


FIGURE 3.13 Illustration of one-dimensional correlation and convolution.

Linear Spatial Filtering(cont.)

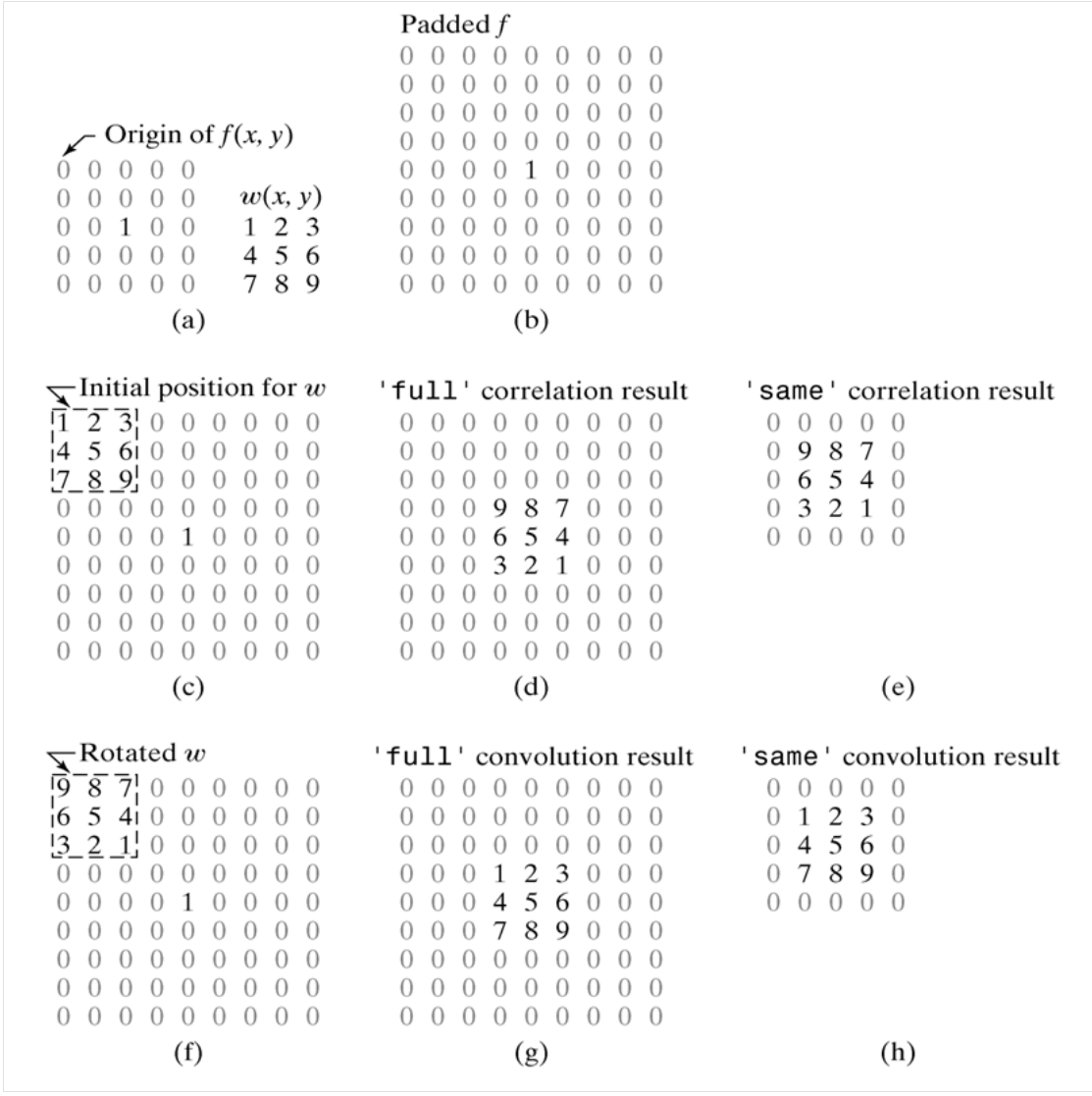


FIGURE 3.14 Illustration of two-dimensional correlation and convolution. The 0s are shown in gray to simplify viewing.

Linear Spatial Filtering(cont.)

- The toolbox implements linear spatial filtering using function imfilter, which has the following syntax:

  g = imfilter(f, w, filtering_mode, boundary_options, size_options)

  where f is the input image, w is the filter mask, g is the filtered result, and the other parameters are summarized in Table 3.2.

Options	Description
Filtering Mode	
`'corr'`	Filtering is done using correlation (see Figs. 3.13 and 3.14). This is the default.
`'conv'`	Filtering is done using convolution (see Figs. 3.13 and 3.14).
Boundary Options	
`P`	The boundaries of the input image are extended by padding with a value, `P` (written without quotes). This is the default, with value 0.
`'replicate'`	The size of the image is extended by replicating the values in its outer border.
`'symmetric'`	The size of the image is extended by mirror-reflecting it across its border.
`'circular'`	The size of the image is extended by treating the image as one period a 2-D periodic function.
Size Options	
`'full'`	The output is of the same size as the extended (padded) image (see Figs. 3.13 and 3.14).
`'same'`	The output is of the same size as the input. This is achieved by limiting the excursions of the center of the filter mask to points contained in the original image (see Figs. 3.13 and 3.14). This is the default.

Linear Spatial Filtering(cont.)

- Each element of the filtered image is computed using double-precision, floating-point arithmetic. However, imfilter converts the output image to the same class of the input.Therefore, if input is an integer array, then output elements that exceed the range of the integer type are truncated, and fractional values are rounded. If more precision is desired in the result, then should be converted to class double by using im2double or double before using imfilter.
  - f=imread('Fig0315(a)(original_test_pattern).tif');
  - f=im2double(f);
  - w=ones(31);
  - gd = imfilter(f,w);
  - imshow(gd,[])

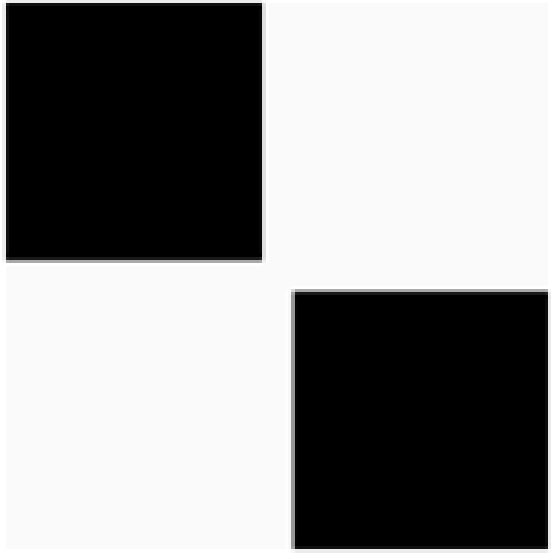
Linear Spatial Filtering(cont.)

- >>gr = imfilter(f, w, 'replicate');
- >>figure, imshow(gr, [ ])
- >>gc = imfilter(f, w, 'circular');
- >>figure, imshow(gc, [ ])

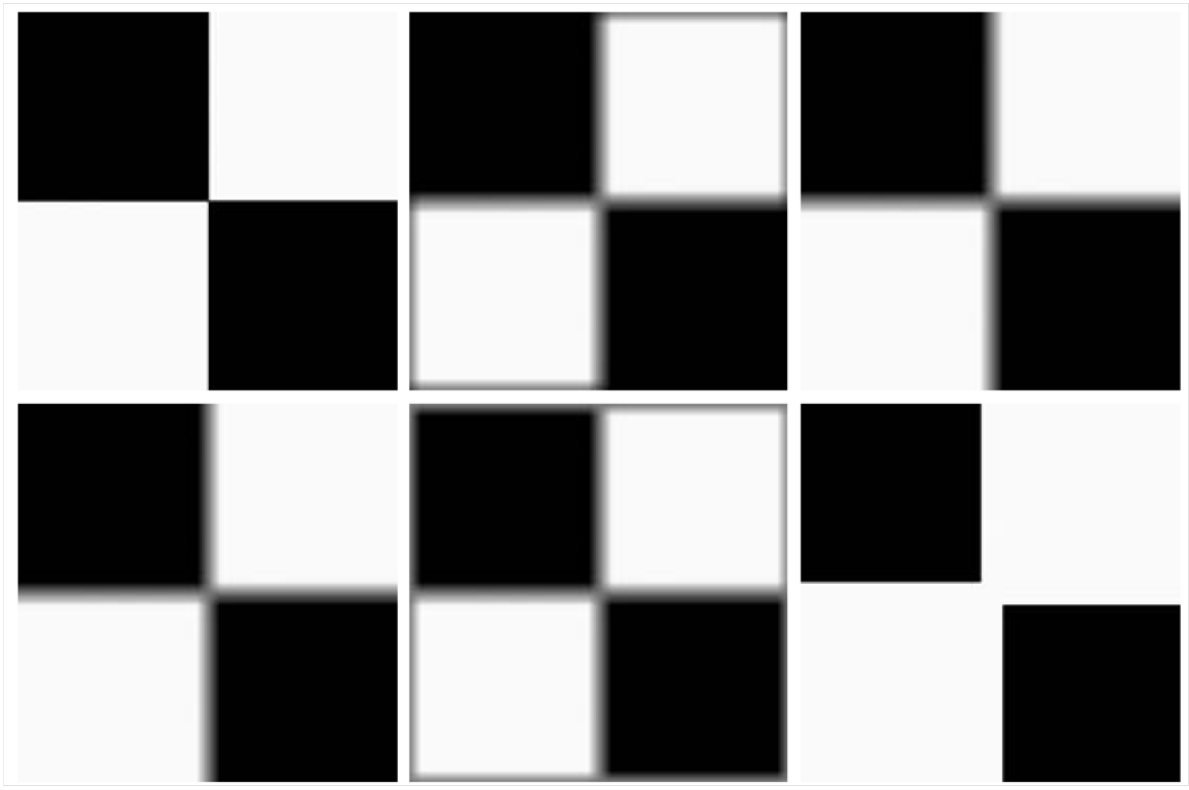
Please help me choose the corresponding output!!

Linear Spatial Filtering(cont.)

- f=imread('Fig0315(a)(original_test_pattern).tif');
- w=ones(31);
- gd = imfilter(f,w);
- imshow(gd,[])



Linear Spatial Filtering(cont.)



a b c
d e f

FIGURE 3.15
(a) Original image. (b) Result of using `imfilter` with default zero padding. (c) Result with the `'replicate'` option. (d) Result with the `'symmetric'` option. (e) Result with the `'circular'` option. (f) Result of converting the original image to class `uint8` and then filtering with the `'replicate'` option. A filter of size 31×31 with all 1s was used throughout.

Linear Spatial Filtering(cont.)

- The toolbox supports a number of predefined 2-D linear spatial filters, obtained by using function fspecial, which generates a filter mask, w, using the syntax

  w = fspecial('type', parameters)

  where 'type' specifies the filter type, and parameters further define the specified filter.

Type	Syntax and Parameters
'average'	fspecial('average', [r c]). A rectangular averaging filter of size r × c. The default is 3 × 3. A single number instead of [r c] specifies a square filter.
'disk'	fspecial('disk', r). A circular averaging filter (within a square of size 2r + 1) with radius r. The default radius is 5.
'gaussian'	fspecial('gaussian', [r c], sig). A Gaussian lowpass filter of size r × c and standard deviation sig (positive). The defaults are 3 × 3 and 0.5. A single number instead of [r c] specifies a square filter.
'laplacian'	fspecial('laplacian', alpha). A 3 × 3 Laplacian filter whose shape is specified by alpha, a number in the range [0, 1]. The default value for alpha is 0.5.
'log'	fspecial('log', [r c], sig). Laplacian of a Gaussian (LoG) filter of size r × c and standard deviation sig (positive). The defaults are 5 × 5 and 0.5. A single number instead of [r c] specifies a square filter.
'motion'	fspecial('motion', len, theta). Outputs a filter that, when convolved with an image, approximates linear motion (of a camera with respect to the image) of len pixels. The direction of motion is theta, measured in degrees, counterclockwise from the horizontal. The defaults are 9 and 0, which represents a motion of 9 pixels in the horizontal direction.
'prewitt'	fspecial('prewitt'). Outputs a 3 × 3 Prewitt mask, wv, that approximates a vertical gradient. A mask for the horizontal gradient is obtained by transposing the result: wh = wv'.
'sobel'	fspecial('sobel'). Outputs a 3 × 3 Sobel mask, sv, that approximates a vertical gradient. A mask for the horizontal gradient is obtained by transposing the result: sh = sv'.
'unsharp'	fspecial('unsharp', alpha). Outputs a 3 × 3 unsharp filter. Parameter alpha controls the shape; it must be greater than 0 and less than or equal to 1.0; the default is 0.2.

Nonlinear Spatial Filtering

- Nonlinear spatial filtering usually uses a neighborhood too, but some other mathematical operations are used. For example, letting the response at each center point be equal to the maximum pixel value in its neighborhood is a nonlinear filtering operation.
- Another basic difference is that the concept of a mask is not as prevalent in nonlinear processing.
- The toolbox provides two functions for performing general nonlinear filtering: nlfilter and colfilt.
- The former performs operations directly in 2-D. Use the command open nlfilter to see the source code.
- colfilt organizes the data in the form of columns. Requires more memory, but executes significantly faster than nlfilter.

Nonlinear Spatial Filtering(cont.)

- Given an input image, f, of size $M \times N$ and a neighborhood of size $m \times n$, function colfilt generates a matrix, call it A, of maximum size $mn \times MN$.
- each column corresponds to the pixels encompassed by the neighborhood centered at a location in the image.
- For example, the first column corresponds to the pixels encompassed by the neighborhood when its center is located at the top, leftmost point in f.
- The former performs operations directly in 2-D. Use the command open nlfilter to see the source code.
- colfilt organizes the data in the form of columns. Requires more memory, but executes significantly faster than nlfilter.

Nonlinear Spatial Filtering(cont.)

- The syntax of function colfilt is

  g = colfilt(f, [m n], 'sliding', @fun, parameters)

  - where, m and n are the dimensions of the filter region
  - 'sliding' indicates that the process is one of sliding the region from pixel to pixel in the input image f
  - @fun references a function, which we denote arbitrarily as fun
  - parameters indicates parameters (separated by commas) that may be required by function fun.
- Because of the way in which matrix A is organized, function fun must operate on each of the columns of A individually and return a row vector, g, containing the results for all the columns.
- The kth element of g is the result of the operation performed by fun on the kth column of A.

Nonlinear Spatial Filtering(cont.)

- When using colfilt, the input image must be padded explicitly before filtering. For this we use function padarray, which, for 2-D functions, has the syntax

  fp = padarray(f, [r c], method, direction)

  where f is the input image, fp is the padded image, [r c] gives the number of rows and columns by which to pad f, and method and direction are as explained in Table 3.3.

TABLE 3.3 Options for function `padarray`.

Options	Description
Method	
`'symmetric'`	The size of the image is extended by mirror-reflecting it across its border.
`'replicate'`	The size of the image is extended by replicating the values in its outer border.
`'circular'`	The size of the image is extended by treating the image as one period of a 2-D periodic function.
Direction	
`'pre'`	Pad before the first element of each dimension.
`'post'`	Pad after the last element of each dimension.
`'both'`	Pad before the first element and after the last element of each dimension. This is the default.

Sharpening Spatial Filters

- The principal objective of sharping is to highlight fine detail in an image or to enhance detail that has been blurred.
- Sharping is generally accomplished by spatial differentiation.
- The derivatives of a digital function are defined in terms of differences.
- For a first derivative, we require that it must satisfy:
  1. must be zero in flat segments(areas of constant gray-level values)
  2. must be nonzero at the onset of a gray-level step or ramp
  3. must be nonzero along ramps
- For a second derivative, we require that it must satisfy:
  1. must be zero in flat segments(areas of constant gray-level values).
  2. must be nonzero at the onset and end of a gray-level step or ramp.
  3. must be zero along ramps of constant slope.

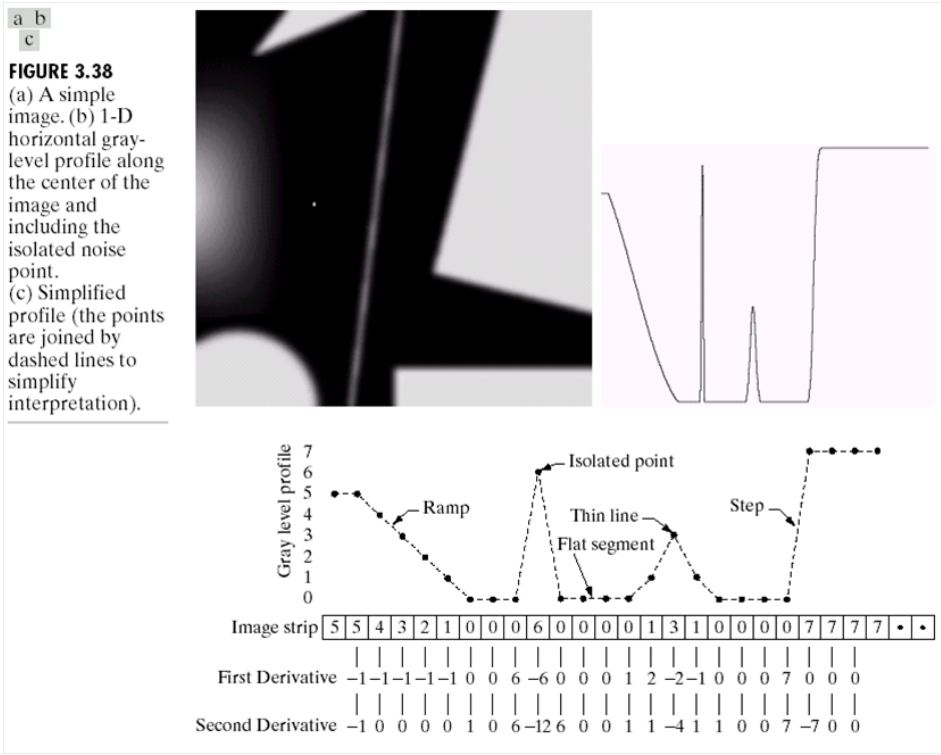
Sharpening Spatial Filters(cont.)

- A basic definition of the first-order derivative of a one-dimensional function $f(x)$ is the difference

$$\frac{\partial f}{\partial x} = f(x+1) - f(x)$$

- Similarly, we define the second derivative as

$$\frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x)$$



a b c

FIGURE 3.38 (a) A simple image. (b) 1-D horizontal gray-level profile along the center of the image and including the isolated noise point. (c) Simplified profile (the points are joined by dashed lines to simplify interpretation).

Laplacian Filter

- Laplacian function is defined as

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} = [f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1)] - 4f(x,y)$$

- Function fspecial('laplacian', alpha) implements a more general Laplacian mask:

$$\begin{pmatrix} \frac{\alpha}{1+\alpha}, & \frac{1-\alpha}{1+\alpha}, & \frac{\alpha}{1+\alpha} \\ \frac{1-\alpha}{1+\alpha}, & \frac{-4}{1+\alpha}, & \frac{1-\alpha}{1+\alpha} \\ \frac{\alpha}{1+\alpha}, & \frac{1-\alpha}{1+\alpha}, & \frac{\alpha}{1+\alpha} \end{pmatrix}$$

- We now proceed to enhance the image in Fig. 3.16(a) using the Laplacian. This image is a mildly blurred image of the North Pole of the moon. Enhancement in this case consists of sharpening the image, while preserving as much of its gray tonality as possible.

Image Enhancement with Laplacian Filters

- $g(x,y) = f(x,y) - \nabla^2 f(x,y)$

a b
c d

FIGURE 3.16
(a) Image of the North Pole of the moon.
(b) Laplacian filtered image, using `uint8` formats.
(c) Laplacian filtered image obtained using `double` formats.
(d) Enhanced result, obtained by subtracting (c) from (a).
(Original image courtesy of NASA.)

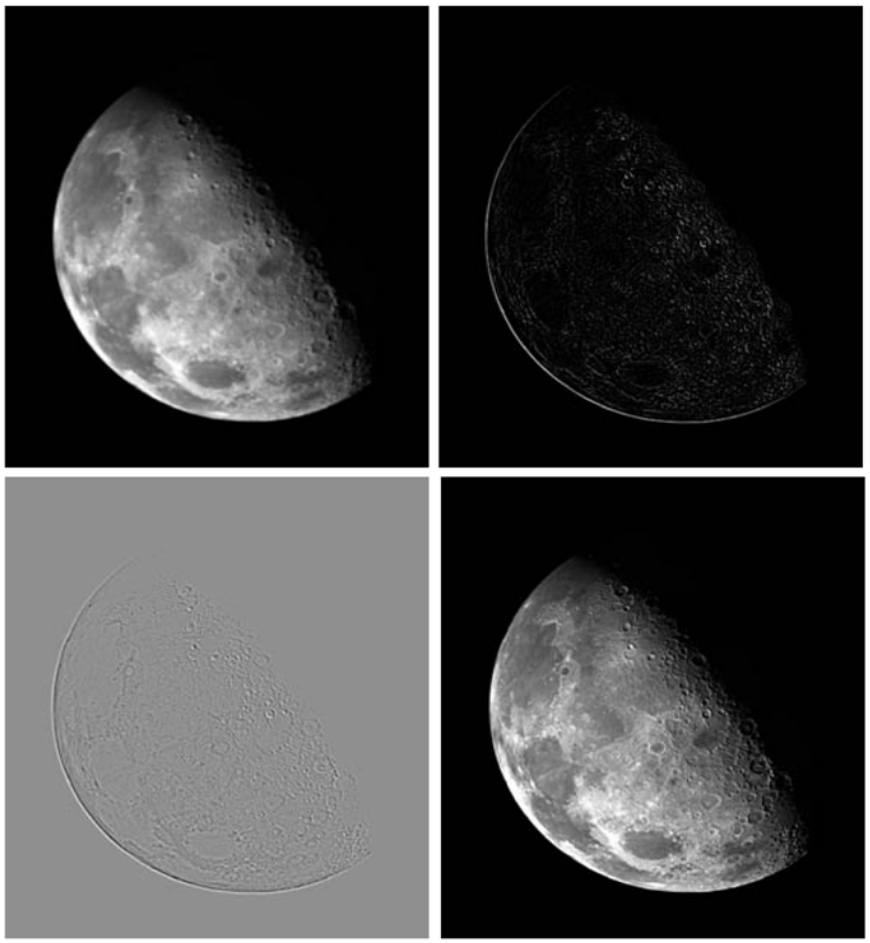
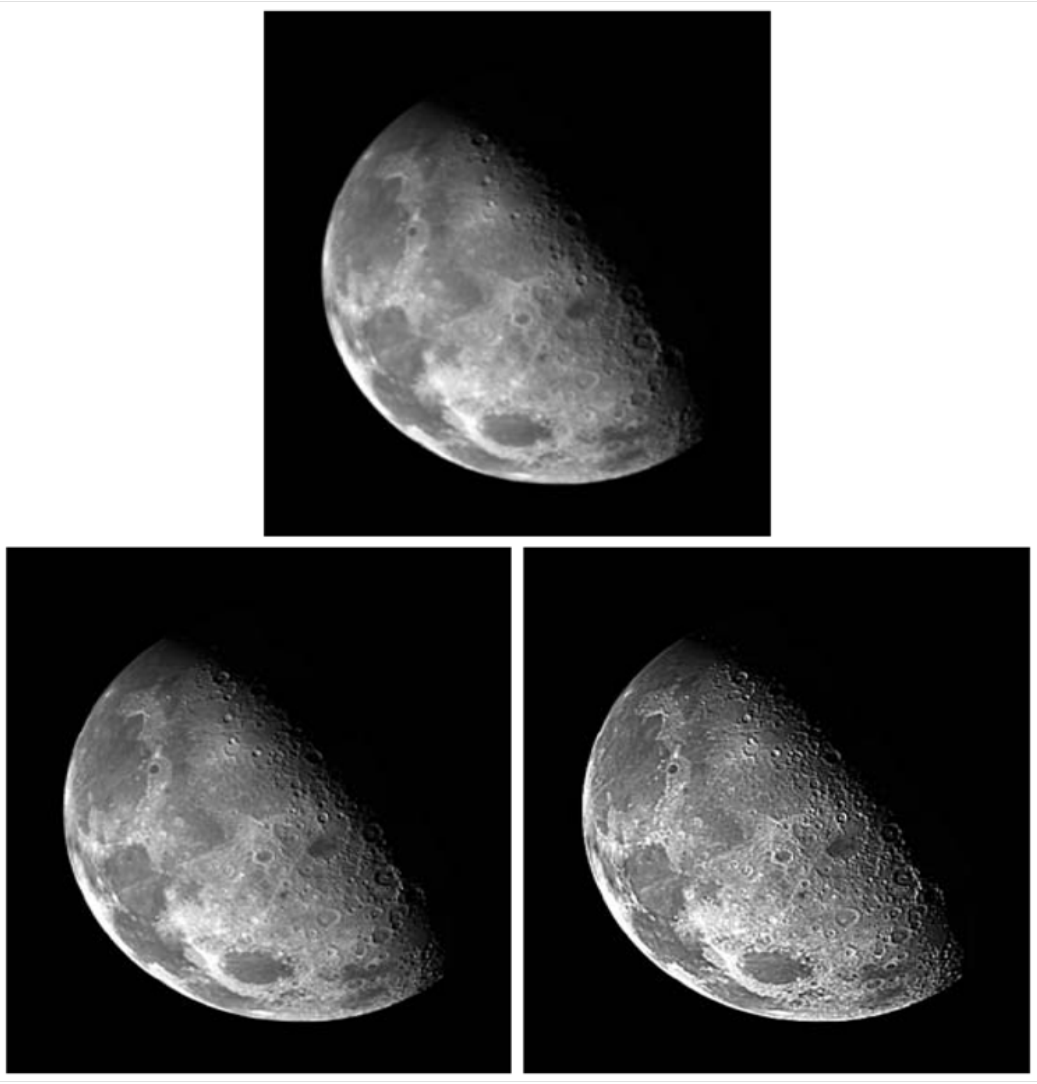


Image Enhancement with Laplacian Filters(cont.)

- Enhancement problems often require the specification of filters beyond those available in the toolbox. The Laplacian is a good example. The toolbox supports a 3×3 Laplacian filter with a -4 in the center. Usually, sharper enhancement is obtained by using the 3×3 Laplacian filter that has a -8 in the center and is surrounded by 1s, as discussed earlier.

```
>>f = imread('Fig0316(a)(moon).tif');
>>w4 = fspecial('laplacian', 0);
>>w8 = [1 1 1; 1 -8 1; 1 1 1];
>>f = im2double(f);
>>g4 = f-imfilter(f, w4, 'replicate');
>>g8 = f-imfilter(f, w8, 'replicate');
>>imshow(f)
>>figure, imshow(g4)
>>figure, imshow(g8)
```

Image Enhancement with Laplacian Filters(cont.)



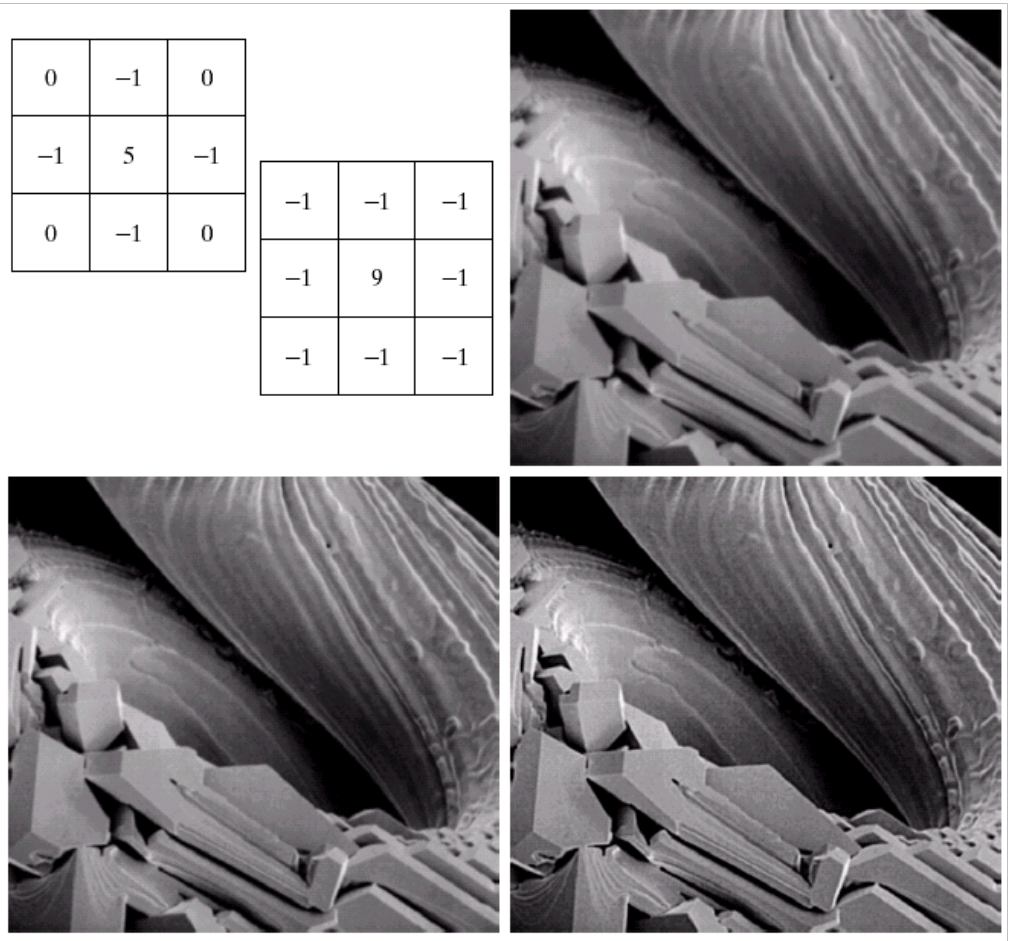
a
b c

FIGURE 3.17 (a) Image of the North Pole of the moon. (b) Image enhanced using the Laplacian filter `'laplacian'`, which has a −4 in the center. (c) Image enhanced using a Laplacian filter with a −8 in the center.

Image Enhancement with Laplacian Filters(cont.)

Simplification:

$$g(x,y) = f(x,y) - \nabla^2 f(x,y) = 5f(x,y) - [f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1)]$$



a b c
d e

FIGURE 3.41 (a) Composite Laplacian mask. (b) A second composite mask. (c) Scanning electron microscope image. (d) and (e) Results of filtering with the masks in (a) and (b), respectively. Note how much sharper (e) is than (d). (Original image courtesy of Mr. Michael Shaffer, Department of Geological Sciences, University of Oregon, Eugene.)

Unsharp masking and high-boost filtering

- Unsharp masking is implemented by subtract a blurred version of an image from the image itself, i.e.,

$$f_s(x, y) = f(x, y) - \bar{f}(x, y)$$

where $f_s(x, y)$ denotes the sharpened image and $\bar{f}(x, y)$ is a blurred version of $f(x, y)$

- A slight further generalizaiton of unsharp masking is called high-boost filtering, and a high-boost filtered image, $f_{hb}(x, y)$, is defined as

$$f_{hb}(x, y) = Af(x, y) - \bar{f}(x, y)$$

where $A \geq 1$, and $\bar{f}(x, y)$ is a blurred version of $f(x, y)$

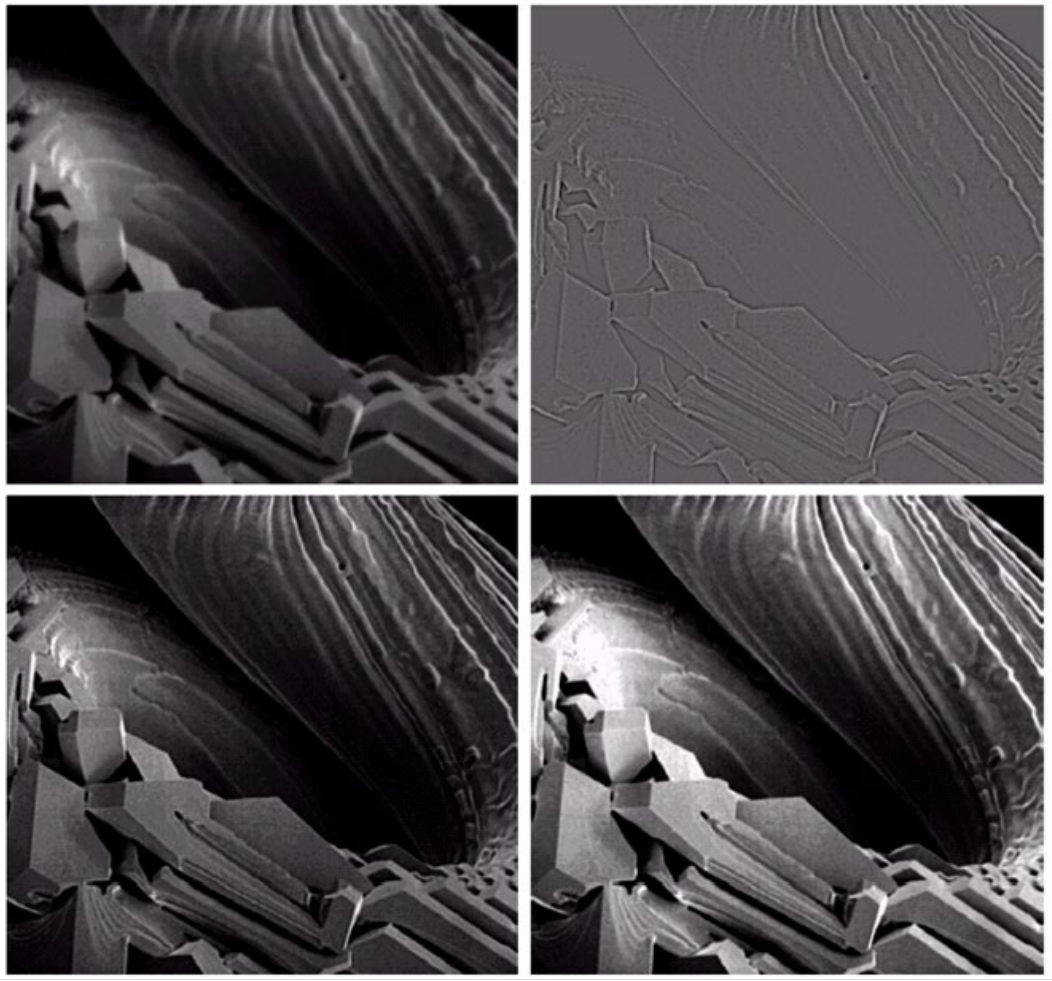
- Further, we can obtain

$$f_{hb}(x, y) = (A - 1)f(x, y) + f(x, y) - \bar{f}(x, y)$$
$$f_{hb}(x, y) = (A - 1)f(x, y) + f_s(x, y)$$

- If we use the Laplacian to compute $f_s(x, y)$, we have

$$f_{hb}(x, y) = (A - 1)f(x, y) - \nabla^2 f(x, y)$$
$$\text{or } f_{hb}(x, y) = (A - 1)f(x, y) + \nabla^2 f(x, y)$$

High-Boost Filtering with Laplacian Filters(cont.)



a b
c d

FIGURE 3.43
(a) Same as Fig. 3.41(c), but darker.
(b) Laplacian of (a) computed with the mask in Fig. 3.42(b) using $A = 0$.
(c) Laplacian enhanced image using the mask in Fig. 3.42(b) with $A = 1$. (d) Same as (c), but using $A = 1.7$.

Image Enhancement with the Gradient

a
b c
d e

FIGURE 3.44
A 3 × 3 region of an image (the z's are gray-level values) and masks used to compute the gradient at point labeled z_5. All masks coefficients sum to zero, as expected of a derivative operator.

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

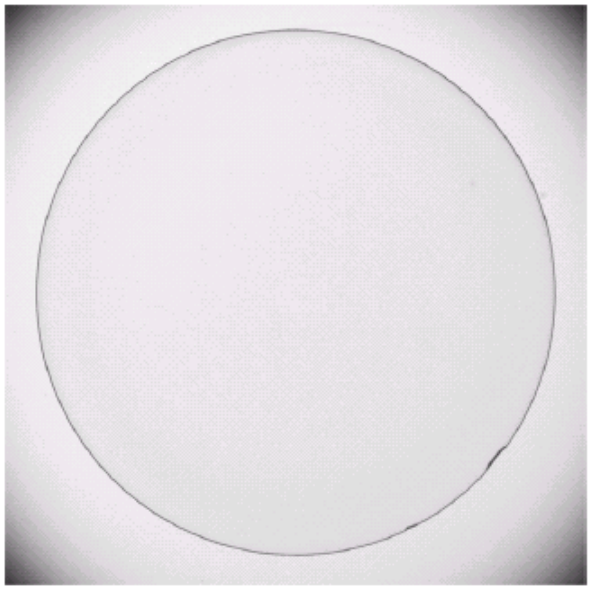
−1	0
0	1

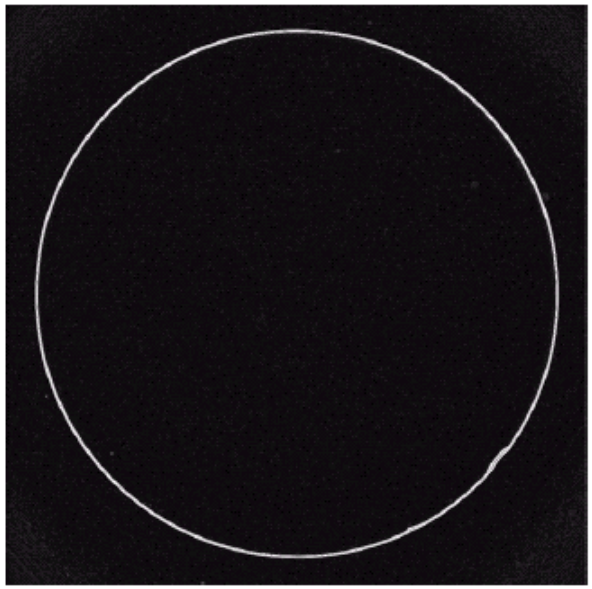
0	−1
1	0

−1	−2	−1
0	0	0
1	2	1

−1	0	1
−2	0	2
−1	0	1

Image Enhancement with the Gradient(cont.)





a b

FIGURE 3.45 Optical image of contact lens (note defects on the boundary at 4 and 5 o'clock). (b) Sobel gradient. (Original image courtesy of Mr. Pete Sites, Perceptics Corporation.)

Nonlinear Spatial Filters

- A commonly-used tool for generating nonlinear spatial filters in IPT is function ordfilt2, which generates order-statistic filters (also called rank filters).
- The syntax of function ordfilt2 is

  g = ordfilt2(f, order, domain)

  This function creates the output image g by replacing each element of f by the order-th element in the sorted set of neighbors specified by the nonzero elements in domain.
- For example, to implement *a min filter* (order 1) of size $m \times n$ we use the syntax

  g = ordfilt2(f, 1, ones(m, n))

  g = ordfilt2(f, m*n, ones(m, n))

  g = ordfilt2(f, median(1:m*n), ones(m, n))

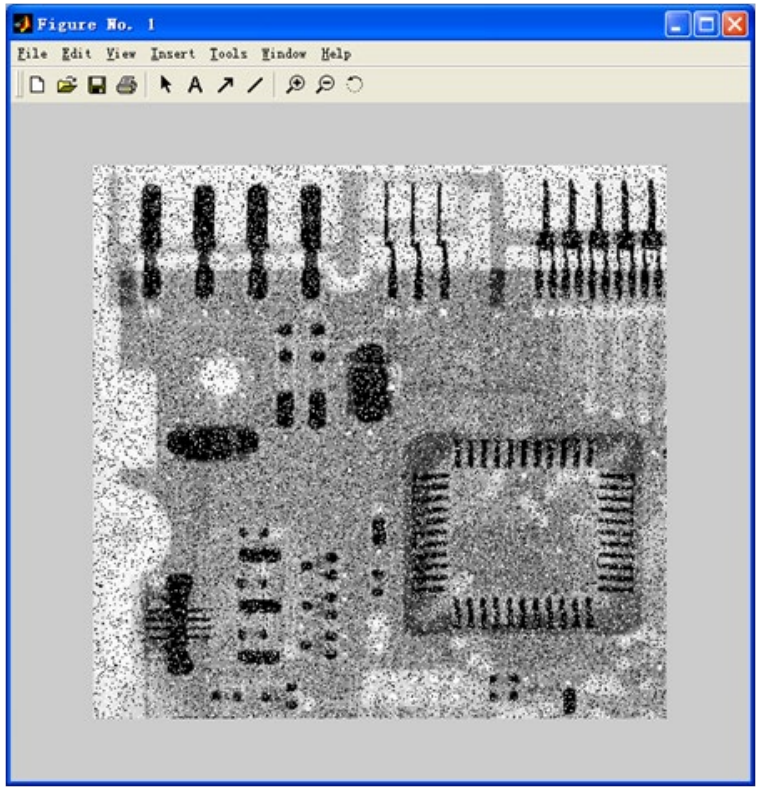
Nonlinear Spatial Filters-Medium Filter

- The best-known order-statistic filter in digital image processing is the median filter
- Because of its practical importance, the toolbox provides a specialized implementation of the 2-D median filter:
  - g = medfilt2(f, [m n], padopt)

    where the tuple [m n] defines a neighborhood of size m×n over which the median is computed, and padopt specifies one of three possible border padding options: 'zeros' (the default), 'symmetric' in which f is extended symmetrically by mirror-reflecting it across its border, and 'indexed', in which f is padded with 1s if it is of class double and with 0s otherwise.

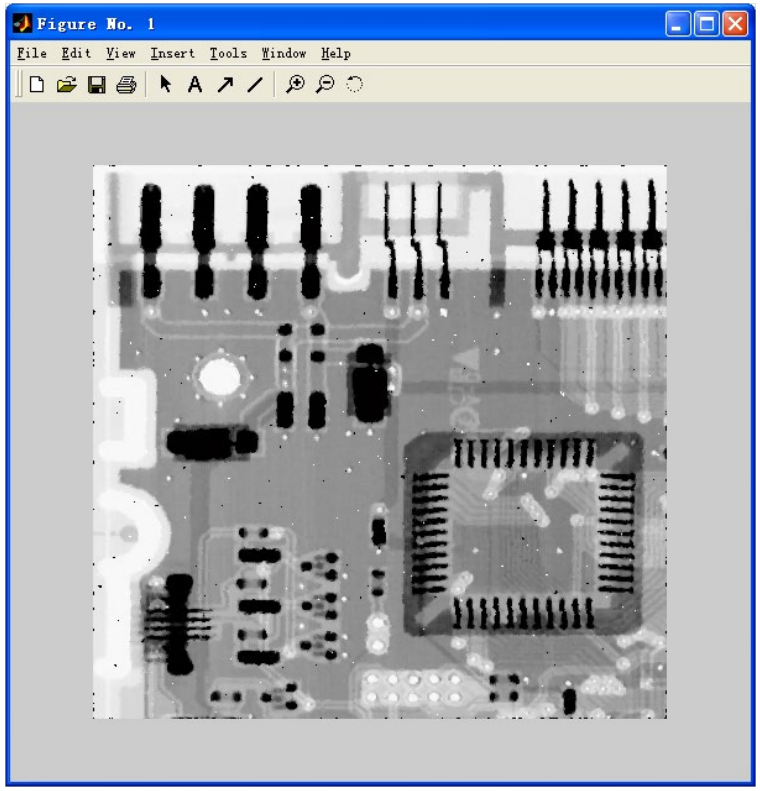
Nonlinear Spatial Filters(cont.)

- Median filtering is a useful tool for reducing salt-and-pepper noise in an image.
  - f=imread('Fig0318(a)(ckt-board-orig).tif');
  - fn=imnoise(f, 'salt & pepper', 0.2);
  - imshow(fn)



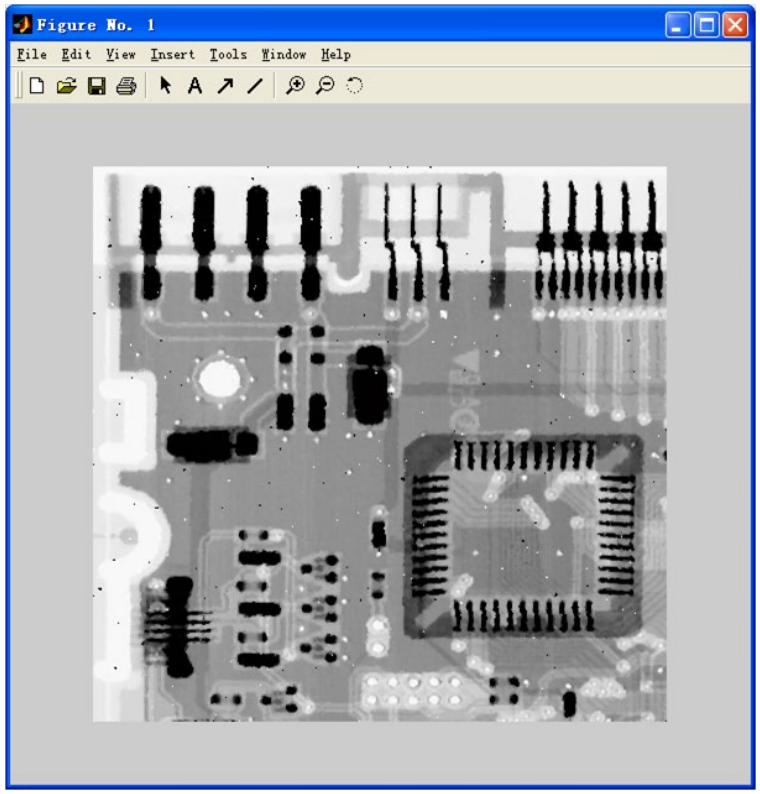
Nonlinear Spatial Filters(cont.)

- gm = medfilt2(fn);
- imshow(gm)

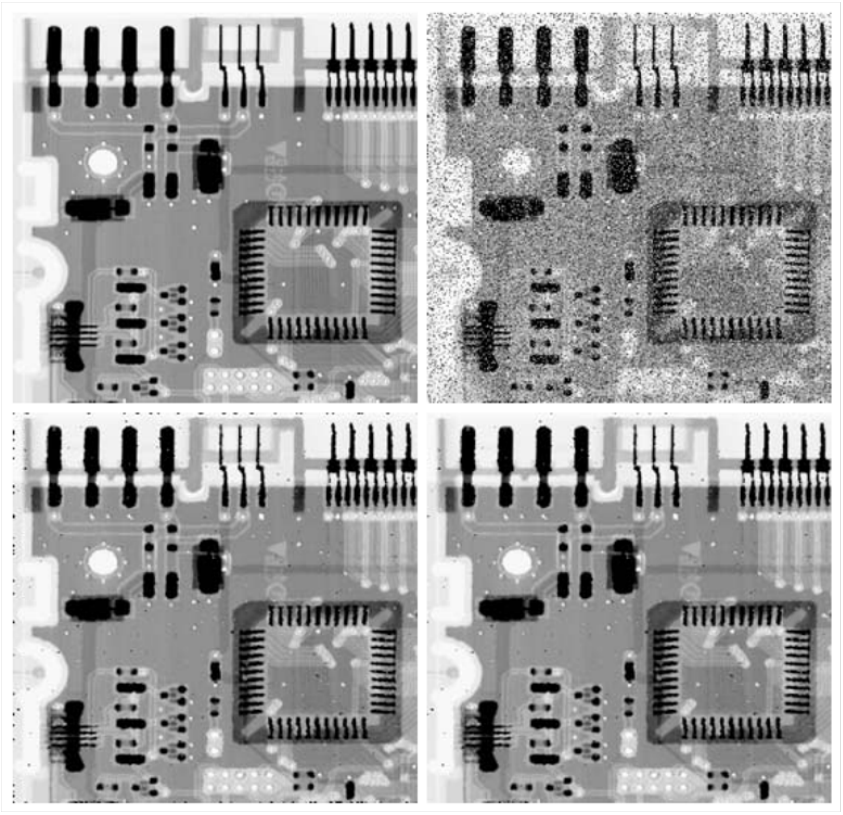


Nonlinear Spatial Filters(cont.)

- gms = medfilt2(fn, 'symmetric');
- imshow(gms)



Nonlinear Spatial Filters(cont.)



a b
c d

FIGURE 3.18 Median filtering, (a) X-ray image. (b) Image corrupted by salt-and-pepper noise. (c) Result of median filtering with `medfilt2` using the default settings. (d) Result of median filtering using the `'symmetric'` image extension option. Note the improvement in border behavior between (d) and (c). (Original image courtesy of Lixi, Inc.)